

AN IRREDUCIBLE FIRST PRINCIPLES MANUAL

The Vibe Coder's Guide to LLMs

How Transformers Actually Work: From Basic High School Math to Scaled Self-Attention and Reasoning AIs.



2026 EDITION

The Vibe Coder's Guide to LLMs

How Transformers Actually Work: From Basic Math to Reasoning AI

Written by Nilesh & Mike (Nilesh's personal AI assistant)

Chapter 0: The Master Architecture Blueprint

The Complete End-to-End Mental Model of Modern Transformers

| The Bird's-Eye View: How an LLM Actually Works

Before we dive into the mathematical machinery of individual equations and matrices, you need a high-level mental map.

Modern artificial intelligence (from ChatGPT and Claude to Gemini and DeepSeek) can feel mysterious from the outside. But mechanically, every single modern Large Language Model is built on one unified architecture: **The Decoder-Only Transformer**. At its core, a Transformer is simply an **information assembly line**. Think of it like an ultra-fast conveyor belt running through an automated factory:

1. **Raw Words** arrive at the factory entrance.
2. They are stamped into **Lists of Numbers (Vectors)**.
3. They are loaded onto a moving conveyor belt called the **Residual Stream**.
4. As they travel along the belt, multiple worker stations read from them, compare them with neighboring words, and write new factual insights back onto the belt.
5. At the factory exit, the final numbers are translated back into ordinary words, one token at a time.

Below is the complete architectural map showing every single station your words travel through.

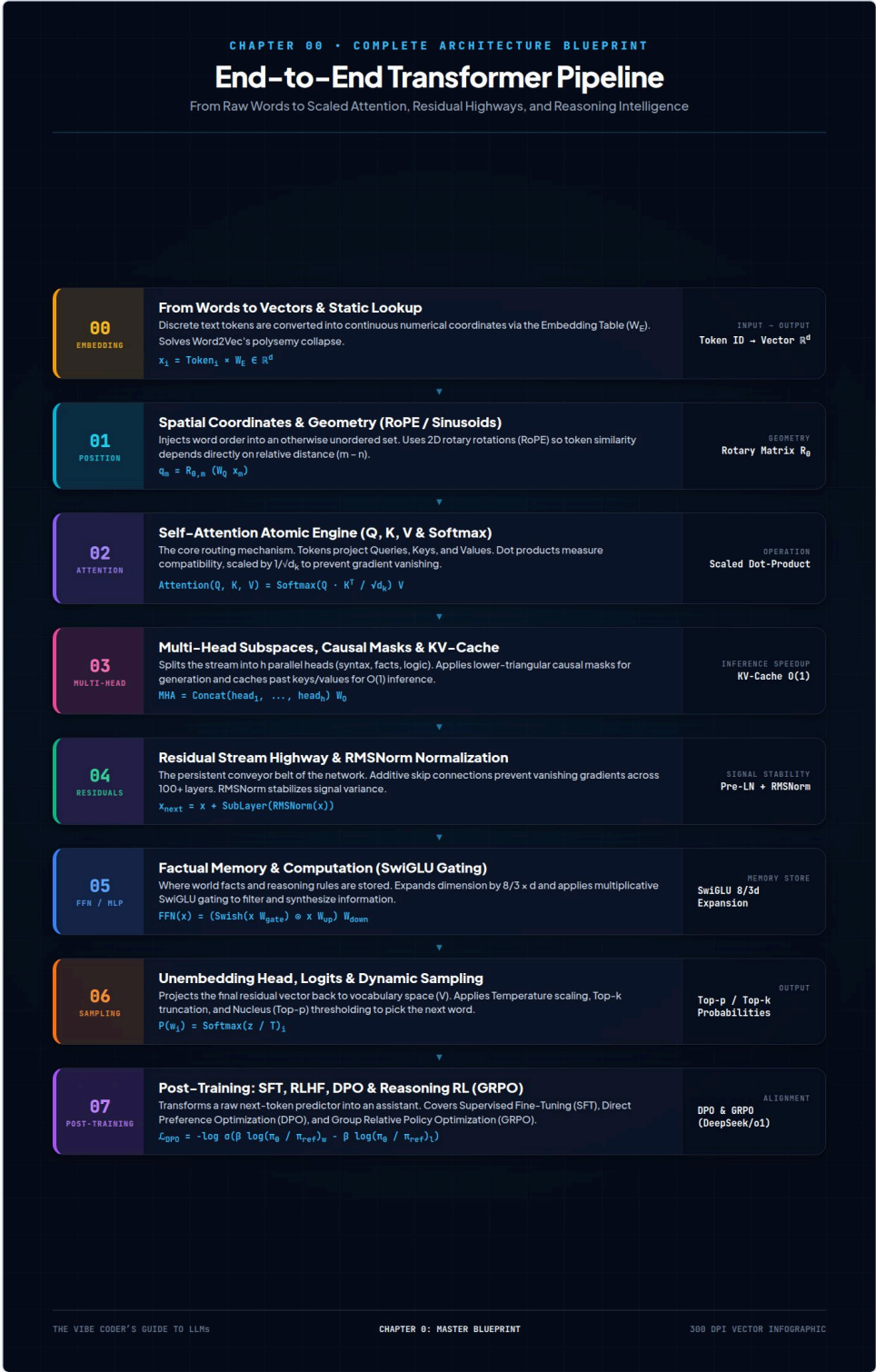


Figure 0.0: The Complete End-to-End Transformer Architecture Blueprint

The 8 Stations of the Transformer Pipeline

Every subsequent module in this book focuses on exactly one station of this factory floor. Here is how they connect together:

⊕ 0: Discrete Words to Continuous Vectors (The Embedding Table)

Computers cannot understand letters or words; they can only calculate with numbers. Station 0 takes raw words (like "bank" or "apple") and looks up their starting coordinates in a massive vocabulary lookup table called the **Embedding Matrix** (W_E). Older models (like Word2Vec) assigned one rigid coordinate to each word, which meant the financial word "bank" and the river "bank" got mashed together into a useless average. Station 0 gives every word a flexible launching pad so later layers can bend its meaning to fit the sentence.

⊕ 1: Injecting Spatial Coordinates (RoPE & Sinusoids)

The core Transformer engine is naturally blind to word order. To the raw math, *"dog bites man"* looks identical to *"man bites dog"*. Station 1 gives every word a spatial clock. In modern models, we use **Rotary Position Embeddings (RoPE)**. Instead of simply adding position numbers, RoPE mathematically rotates the word vectors in 2D pairs like the hands of a clock. As a result, how two words interact depends purely on their relative distance from each other.

⊕ 2: The Self-Attention Engine (Queries, Keys & Values)

This is the communication hub of the entire model. Every word emits three specialized vectors: **Query (Q)** (what this word is searching for), **Key (K)** (what this word contains), and **Value (V)** (the actual substantive information to pass along). By computing the dot product between Queries and Keys, the model measures how relevant every word is to every other word. It scales the numbers by $1/\sqrt{d_k}$ (to keep the math stable) and passes them through **Softmax** to convert them into percentages that sum to 100%. Finally, it takes a weighted mixture of the Values.

⊕ 3: Multi-Head Subspaces, Causal Masking & Fast KV-Cache

Language is too complex for a single perspective. Station 3 splits the attention process into multiple parallel **Attention Heads** (e.g., 32 or 64 heads). One head tracks grammar, another tracks pronoun owners, and another tracks dates. It also enforces **Causal Masking**—preventing words from peeking at future tokens during generation. During chat generation, it activates the **KV-Cache**, storing past keys and values in GPU memory so the model only calculates the newest token ($O(1)$ inference time instead of re-reading the entire book every second).

⊕ 4: The Residual Stream Highway & RMSNorm

Modern models are deep—often stacking 80 to 128 transformer layers on top of each other. If numbers were multiplied through 100 layers in a row, they would either shrink to zero (vanishing gradients) or blow up to infinity (exploding gradients). The **Residual Stream** is an uninterrupted highway running through the entire network. Instead of overwriting the token vector, each layer merely *adds* its small update to the stream ($x_{\text{next}} = x + \text{Update}$). To prevent numbers from growing unchecked, **RMSNorm** rescales the vector variance before each operation.

⊕ 5: Factual Memory & Nonlinear Computation (The SwiGLU MLP)

While Attention allows tokens to talk to *each other*, the **Feed-Forward Network (FFN)** allows tokens to consult the model's internal encyclopedic memory. The FFN expands the vector's size (usually by a factor of $8/3 \times d$), applies a multiplicative gate (**SwiGLU**), and compresses it back down. This is where factual knowledge (e.g., *"Paris is the capital of France"*, Python syntax rules, mathematical formulas) is permanently stored.

⊕ 6: Unembedding Head, Logits & Dynamic Sampling

At the end of 100+ layers, our token vector on the conveyor belt has been enriched with context, grammar, and facts. Station 6 projects this vector back across all 128,000 words in the vocabulary using the **Unembedding Matrix (W_U)**, producing raw scores called **Logits**. These scores are converted into probabilities using **Temperature** (controlling creativity), **Top-k** (keeping only top k candidates), and **Nucleus Top-p** (sampling from the cumulative probability set).

⊕ 7: Post-Training, Alignment & Reasoning RL

The raw pre-trained model is simply a brilliant auto-complete engine. Station 7 transforms the raw predictor into a helpful, truthful, reasoning assistant. We explore **Supervised Fine-Tuning (SFT)**, **Direct Preference Optimization (DPO)**, and modern **Reasoning RL (GRPO)** as used in frontier reasoning models like DeepSeek-R1 and OpenAI o1.



▮ The End-to-End Pipeline Summary Table

Stage	Name	Input → Output	What Breaks If We Omit It?
00			

****Embeddings**** | Token ID → Vector $\mathbb{R}^d$ | Words remain text strings; math cannot operate on them. | | ****01**** | ****Position (RoPE)**** | Vector → Vector with Rotated Coordinates | Model becomes an unordered bag of words ("dog bites man" = "man bites dog"). | | ****02**** | ****Self-Attention**** | Context Vectors → Contextualized Mix | Words cannot communicate; word meanings stay isolated and naive. | | ****03**** | ****Multi-Head & KV-Cache**** | Multiple Heads → Fused Subspaces | Model can only track one semantic relationship at a time; inference slows to a crawl. | | ****04**** | ****Residuals & RMSNorm**** | Stream Vector → Stabilized Stream | Deep networks (30+ layers) collapse during training due to vanishing gradients. | | ****05**** | ****FFN (SwiGLU)**** | Vector → Factually Enriched Vector | Model has no internal memory store; cannot recall historical facts or code patterns. | | ****06**** | ****Unembedding & Sampling**** | Vector → Next-Token String | The vector cannot be translated back into human-readable text. | | ****07**** | ****Alignment & Reasoning**** | Raw Predictor → Reasoning AI | Model hallucinates, ignores user instructions, or completes prompts blindly. |

Module 0: The Bridge from Word2Vec

From Static Semantic Dictionaries to Contextual State Spaces

0.1 Word2Vec's Polysemy Collapse

🔍 MOTIVATION & FUNCTIONAL ROLE

In the early days of language models, architectures like Word2Vec and GloVe operated like a simple physical dictionary: every word in the vocabulary was assigned exactly one fixed list of numbers (a spatial coordinate or "vector"). However, human language is highly ambiguous. The word "bank" can mean a financial institution or the side of a river. This ambiguity is called *polysemy*.

If an AI relies on static, unmoving dictionary lookups, it suffers from a "polysemy collapse." It is forced to squish all possible meanings of a word into a single coordinate. The Transformer architecture was invented specifically to solve this. Instead of retrieving a static, rigid meaning from a hard drive, the Transformer dynamically updates the word's numerical coordinate based on the surrounding sentence, allowing the AI to understand context.

📖 DEFINITION & ARCHITECTURE

In older static models, the training process scanned millions of documents and created a single average coordinate for every word. If two meanings were entirely opposite, they would cancel each other out, landing the word in an intermediate vacuum of space that correctly represented nothing at all.

The Transformer architecture decouples the **identity** of the token (what the word literally is) from its **contextual state** (what the word means right now). It starts by retrieving a base dictionary coordinate, but immediately uses Self-Attention layers to look at surrounding words. By calculating how much "affinity" or relevance surrounding words have, the Transformer dynamically pulls and steers the original coordinate toward its exact, situational meaning.

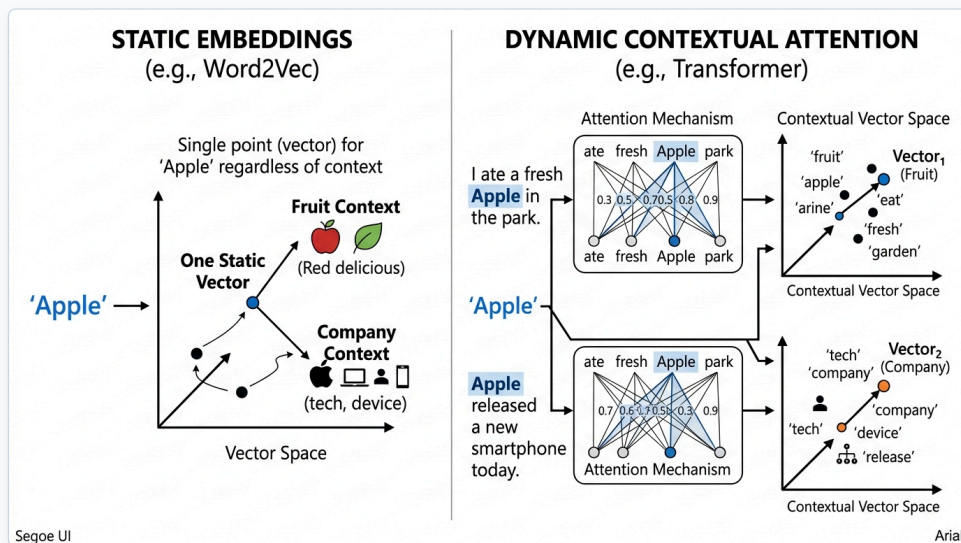


Figure 0.1: Static Word2Vec Context Collapse vs Dynamic Contextual Trajectory

⊕ MATHEMATICAL WORKING & FORMULATION

The Immediate Goal: We must first understand how older models (like Word2Vec) calculated their rigid, unmoving word coordinates by blending all historical meanings together.

$$\mathbf{e}_{\text{static}}(w) \approx \sum_{k=1}^K P(s_k | w) \mathbf{u}_{s_k}$$

The Explanation: In this formula, $\mathbf{e}_{\text{static}}(w)$ represents the final, fixed coordinate (vector) assigned to a specific word w out of the total vocabulary V . The Σ (sigma) symbol simply means "add everything up." We are adding up all the different distinct meanings (denoted as s_k) that the word has in the real world. For each meaning, there is an ideal, perfect numerical coordinate $\mathbf{u}_{s_k}$. We multiply that ideal coordinate by $P(s_k | w)$, which is just a percentage representing how often that specific meaning occurs in text. Simply put, this equation calculates a weighted average. It forces all distinct meanings into one compromised, static number.

The Next Step: To fix this compromise, the Transformer model completely discards the idea of a fixed coordinate. Instead, it creates a dynamic "hidden state" that will evolve as we move deeper into the network. First, we must create the starting point for this dynamic journey.

$$\mathbf{h}_i^{(0)} = \mathbf{e}_i + \mathbf{p}_i$$

The Explanation: Here, $\mathbf{h}_i^{(0)}$ represents the hidden state of a word at position i in a sentence, at layer 0 (the very beginning of the network). To build it, we take the initial static dictionary lookup $\mathbf{e}_i$ (similar to the Word2Vec base) and we mathematically add $\mathbf{p}_i$, which is a positional encoding. Think of $\mathbf{p}_i$ as a numerical time-stamp. Because all words enter the Transformer simultaneously, adding $\mathbf{p}_i$ ensures the network knows exactly where the word physically sits in the sentence sequence.

The Next Step: Now that we have a starting coordinate that knows its position, the Transformer must update this coordinate by gathering context from the other words in the sentence. This happens as we move up through the network's layers.

$$\mathbf{h}_i^{(\ell)} = \mathbf{h}_i^{(\ell-1)} + \sum_{j=1}^N \alpha_{ij}^{(\ell)} (\mathbf{h}_j^{(\ell-1)} W_V^{(\ell)})$$

The Explanation: We are now calculating $\mathbf{h}_i^{(\ell)}$, which is the updated, richer meaning of our word at the current layer ℓ . We start with the word's meaning from the previous layer, $\mathbf{h}_i^{(\ell-1)}$. We then use the addition symbol to mix in new context from all N surrounding words in the sequence. For every surrounding word (indexed by j), we take its previous state $\mathbf{h}_j^{(\ell-1)}$ and multiply it by a learned grid of numbers called the Value matrix, $W_V^{(\ell)}$. This extracts the core underlying concept of that surrounding word. Finally, we multiply that extracted concept by $\alpha_{ij}^{(\ell)}$. This is the crucial "attention weight"—a single decimal number that dictates exactly how much attention word i should pay to word j .

The Next Step: How does the Transformer know what that attention weight should be? It must calculate how highly related the two words are by asking questions and checking answers.

$$\alpha_{ij}^{(\ell)} = \frac{\exp\left(\frac{(\mathbf{h}_i^{(\ell-1)} W_Q^{(\ell)}) (\mathbf{h}_j^{(\ell-1)} W_K^{(\ell)})^\top}{\sqrt{d_k}}\right)}{\sum_{m=1}^N \exp\left(\frac{(\mathbf{h}_i^{(\ell-1)} W_Q^{(\ell)}) (\mathbf{h}_m^{(\ell-1)} W_K^{(\ell)})^\top}{\sqrt{d_k}}\right)}$$

The Explanation: This calculates our specific attention score, $\alpha_{ij}^{(\ell)}$. We take our current word and multiply it by a Query matrix, $W_Q^{(\ell)}$, essentially translating the word into a question: "What context am I looking for?" We take the surrounding word and multiply it by a Key matrix, $W_K^{(\ell)}$, translating it into a tag: "What context do I contain?" The $^\top$ symbol just flips the second list of numbers on its side so we can multiply them together (an operation called a dot product). This multiplication results in a raw match score. We divide by $\sqrt{d_k}$ (the square root of the dimensions of the list, like 64) just to shrink the numbers so they don't blow up and break the math. Finally, the exp (exponential function) on the top and the sum division on the bottom act as a "Softmax" function. This simply turns all the raw match scores into neat percentages that add up to exactly 1.0 (or 100%), ensuring the network distributes its limited attention perfectly across the sentence.

□ Concrete Numerical Toy Walk-Through

Let us prove why Word2Vec fails and the Transformer succeeds using basic high-school numbers. Imagine our vocabulary has words plotted on a simple 2D X-Y grid.

- **The Problem:** The word "bank". In a massive text corpus, 50% of the time it means "Money" (let's say coordinate [0, 10]). The other 50% of the time it means "River" (coordinate [10, 0]).
- **Word2Vec's Failure:** Word2Vec calculates the weighted average.

$$0.5 * [0, 10] + 0.5 * [10, 0] = [5, 5]$$

The final static coordinate for "bank" is [5, 5]. This is halfway between money and a river. It is a meaningless vacuum.

- **The Transformer's Solution:** Let's say the user types: "water bank". The Transformer starts "bank" at the base [5, 5]. But it looks at the surrounding word "water" which sits at [10, 0].

The attention math determines "water" is highly relevant (attention score of 1.0). The network extracts the value of "water" and adds it to update "bank".

Let's say the extracted update vector is [5, -5].

[5, 5] (old bank) + [5, -5] (update from water) = [10, 0]

Through simple addition, the Transformer has dynamically steered "bank" away from the vacuum and perfectly onto the "River" coordinate!

□ Training vs. Inference

- **During Training (Learning the Weights on GPUs):** The model is fed thousands of complete documents simultaneously across massive clusters of GPUs. It processes all words in a sentence at the exact same time (in parallel). The matrices W_Q , W_K , and W_V are actively being updated and improved using calculus (backpropagation) to minimize the model's errors based on a known correct answer (teacher forcing).
- **During Inference (Runtime Chat with ChatGPT/Claude):** The training is over. The matrices W_Q , W_K , and W_V are permanently frozen; they do not learn or change. When a human types a prompt, the model generates words exactly one at a time from left to right (autoregressively). Because it cannot look into the future, it relies on a "KV-Cache"—a memory bank that saves the previous words' Key and Value calculations so they don't have to be recomputed for every new word, ensuring the chat responds instantly.

🌐 REAL-WORLD & NLP BEHAVIORS

- **Financial vs. Ecological Disambiguation:** In the sentence *"The company had to bank on emergency federal credit after the river breached the northern bank"*, the token "bank" occurs twice with entirely distinct meanings. A static embedding model produces identical vectors for both instances, blinding downstream layers. A Transformer attends to "credit" and "federal" for the first occurrence, routing its representation into a finance-associated subspace, while attending to "river" and "breached" for the second, routing it into a geological subspace.
- **Syntactic Category Shift (Noun vs. Verb Conversion):** In *"The complex houses married soldiers"*, the word "houses" acts as an action verb, and "complex" acts as a descriptive adjective. Static models strongly bias "houses" toward its dominant noun meaning (residential buildings), causing the AI to misunderstand the grammar. A Transformer computes strong syntactic affinity between the subject and the verb position, projecting "houses" into an action-oriented subspace that allows it to generate accurate, grammatically correct responses.

0.2 Token Embedding Matrix & The Residual Stream

💡 MOTIVATION & FUNCTIONAL ROLE

Digital computer processors do not understand text, letters, or words; they only understand continuous arrays of numbers. Before any intelligent reasoning can happen, the Transformer must convert discrete categorical tokens (like the word "apple" or "run") into a format the hardware can compute.

To do this, it uses a **Token Embedding Matrix**—a colossal, mathematical dictionary that maps every discrete word to a rich, dense spatial coordinate. More importantly, when this coordinate is created, it is injected into the **Residual Stream**. The Residual Stream is the central communication highway of the Transformer. It runs straight through every layer of the model, allowing attention and feed-forward networks to add new context to the word without ever destroying or overwriting the original word's identity. If this uncorrupted highway didn't exist, a 32-layer deep AI would forget the first word of your prompt by the time it reached the end of the network.

DEFINITION & ARCHITECTURE

Modern language models rely on a vocabulary (often denoted as V) of tens of thousands of subword pieces (like syllables or common words). Each piece is assigned a simple integer ID. The Embedding Matrix stores an equal number of rows, where each row is a dedicated list of numbers (a vector) for that specific ID.

Once a word is converted into its vector form, it enters the Residual Stream. Unlike older sequential models (like LSTMs) which passed data through harsh mathematical transformations that constantly scrambled the coordinates at every step, the Transformer's Residual Stream operates entirely on **addition**. Layers read data from the highway, process it, and simply add their findings back to the stream.

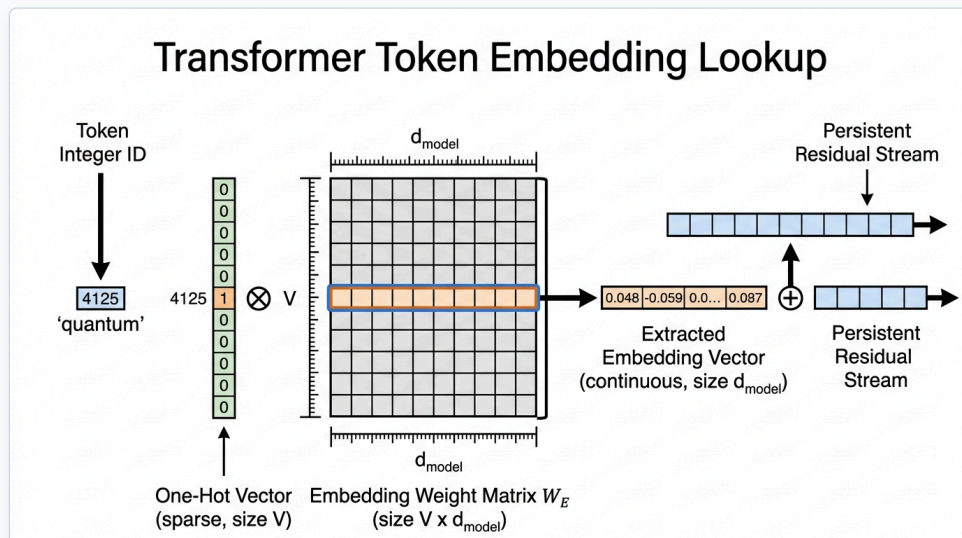


Figure 0.2: Token Embedding Matrix Lookup and Residual Stream Projection

⊞ MATHEMATICAL WORKING & FORMULATION

The Immediate Goal: We must translate a single, discrete integer token ID (like ID: 492) into a dense, continuous list of numbers that the neural network can manipulate.

$$\mathbf{e}_i = \mathbf{t}_i W_E$$

The Explanation: In this formula, $\mathbf{e}_i$ represents the dense, unscaled vector (the raw list of numbers) for the token at position i . To find this, we use $\mathbf{t}_i$, which is a "one-hot indicator vector." Imagine a giant list of zeros as long as the entire vocabulary, with a single '1' placed at the exact index of our word. W_E is the massive learnable Token Embedding Matrix, which contains V rows (the total vocabulary size) and d_{model} columns (the hidden state dimension, like 4096 columns). When we multiply the one-hot vector $\mathbf{t}_i$ by the giant matrix W_E , the math simply plucks out the exact row corresponding to our word. (In real coding, this is optimized as a direct array lookup: $\mathbf{e}_i = W_E[t_i, :]$).

The Next Step: Now that we have our raw dictionary coordinate, we must appropriately size it and give it a position before it merges onto the central highway (the Residual Stream).

$$\mathbf{x}_i^{(0)} = \sqrt{d_{\text{model}}} \cdot \mathbf{e}_i + \mathbf{p}_i$$

The Explanation: Here, $\mathbf{x}_i^{(0)}$ represents the fully prepared, continuous activation vector for our word at layer 0—the very start of the Residual Stream. Before we use it, we take our raw vector $\mathbf{e}_i$ and multiply it by $\sqrt{d_{\text{model}}}$, which is the square root of our total dimensions. We scale it up so that the numerical values are robust and don't get drowned out. Finally, we add $\mathbf{p}_i$, the positional encoding vector, which permanently bakes the word's physical location into the numbers.

The Next Step: The prepared word vector is now traveling down the Residual Stream. It hits the first major checkpoint: the Attention Mechanism, which gathers context from other words and adds it to the stream.

$$\mathbf{x}_i^{(\ell-1)'} = \mathbf{x}_i^{(\ell-1)} + f_{\text{att}}^{(\ell)}(\text{LN}(\mathbf{x}_{1:N}^{(\ell-1)}))_i$$

The Explanation: We are calculating an intermediate state on our highway, denoted by $\mathbf{x}_i^{(\ell-1)'}$. We start with the current state of the highway coming from the previous layer, $\mathbf{x}_i^{(\ell-1)}$. Before processing, the data is passed through LN, a Layer Normalization operator that simply stabilizes the numbers so they don't grow too massive. This stabilized data goes into the multi-head attention sub-layer, $f_{\text{att}}^{(\ell)}$, which looks at all N tokens in the input sequence, calculates how they relate, and outputs a new vector of contextual insight. Crucially, notice the plus sign: this new insight is strictly **added** to the original highway state, perfectly preserving the past.

The Next Step: Finally, this contextualized word must pass through a private reasoning step to digest the new context before layer ℓ is complete.

$$\mathbf{x}_i^{(\ell)} = \mathbf{x}_i^{(\ell-1)'} + f_{\text{MLP}}^{(\ell)}(\text{LN}(\mathbf{x}_i^{(\ell-1)'}))$$

The Explanation: To get the final output vector for layer ℓ , defined as $\mathbf{x}_i^{(\ell)}$, we take our intermediate highway state $\mathbf{x}_i^{(\ell-1)'}$. We stabilize it again with Layer Normalization (LN). Then, it passes through $f_{\text{MLP}}^{(\ell)}$, which is a standard feed-forward neural network (Multi-Layer Perceptron). This acts as a private reasoning chamber for this specific token, allowing it to reflect on the new context it just received. Once again, the output of this reflection is simply **added** back into the stream. The journey continues layer by layer.

□ Concrete Numerical Toy Walk-Through

Let's walk through how a word enters the Residual Stream using basic arithmetic.

- **The Setup:** Imagine a tiny vocabulary of 3 words. Our word is "Cat", which has an ID of 1. The embedding matrix W_E has 3 rows:

Row 0: [1, 2]

Row 1: [3, 4] (This is "Cat")

Row 2: [5, 6]

- **The Lookup:** Our one-hot vector $\mathbf{t}_i$ is [0, 1, 0]. Multiplying this by W_E selects Row 1. So, our raw vector $\mathbf{e}_i$ is [3, 4].
- **Scaling and Position:** Let's say our dimension d_{model} is 4. The square root of 4 is 2.

We scale our vector: $2 * [3, 4] = [6, 8]$.

Now we add a positional vector $\mathbf{p}_i$, let's say [0, 1].

$[6, 8] + [0, 1] = [6, 9]$.

The vector [6, 9] is $\mathbf{x}_i^{(0)}$. It now enters the Residual Stream.

- **The Highway Addition:** At the first layer, the attention mechanism notices the word "Meow" nearby and decides to update "Cat". It generates an update vector of [2, -2].

Because the Residual Stream is additive, we simply do:

$[6, 9]$ (current state) + $[2, -2]$ (attention update) = $[8, 7]$.

The original identity of "Cat" is safely preserved, but altered to reflect its environment!

Training vs. Inference

- **During Training (Learning the Weights on GPUs):** The Embedding Matrix W_E is not static; it is heavily updated. As the model reads billions of words across parallel GPU clusters, it uses the backpropagation algorithm to constantly adjust the numbers inside W_E . If the model learns that "Cat" and "Dog" are used in similar sentences, their specific rows in W_E are mathematically pushed closer together in space.
- **During Inference (Runtime Chat with ChatGPT/Claude):** The Embedding Matrix W_E is locked and frozen as a permanent dictionary. When you chat with the AI, your text is tokenized, and the model executes a lightning-fast array lookup to pull the rows for your words. Because it is generating one new word at a time, the Residual Stream only computes the forward addition math for the very last, newly generated token, recycling the previous tokens' stream states from memory.

🌐 REAL-WORLD & NLP BEHAVIORS

- **Subword Compositionality in BPE Tokenization:** When encountering rare or technical terms like "unconstitutional", modern tokenizers split the string into pieces: ["un", "constitut", "ional"]. W_E maps each sub-piece to an individual row vector. Because W_E was trained on billions of tokens, the vector for "un" points in a mathematical direction corresponding to "negation," while "ional" aligns with "adjective." As these vectors enter the Residual Stream, self-attention adds them together, composing these disjointed pieces into a unified, coherent concept before any deep reasoning occurs.
- **Weight Tying with Output Unembedding:** In many models like GPT-2 and Gemma, the input matrix W_E is mathematically reused at the very end of the network. It is flipped on its side (transposed to $W_U = W_E^T$) to convert the final continuous numbers back into discrete English words. This brilliant engineering trick enforces a strict geometric rule: if two words mean similar things, their input dictionary vectors must be close together, which simultaneously guarantees the network can easily output either word when expressing that specific concept.

Module 1: Spatial & Order Foundations

Attention is Permutation-Invariant

1.1 Permutation Invariance of Set Operations

💡 MOTIVATION & FUNCTIONAL ROLE

- **Importance:** The core mathematical operator of the Transformer—the attention mechanism—simply measures how much words match each other. It operates entirely on multiplying numbers together and adding them up, possessing zero built-in awareness of sequence order, time, or how far apart words are.
- **Functional Role:** This shows why Transformers, in their bare mathematical form, treat sentences as unordered bags of words (like Scrabble tiles dumped on a table). It proves exactly why we are forced to artificially inject "position signals" into the math if we want the model to understand grammar and syntax.
- **LLM Behavior Affected:** If we did not add positional encoding, an AI would calculate the exact same mathematical thought process for "dog bites man" and "man bites dog." It would produce the identical outputs for both, rendering grammatical parsing, logical reasoning, and causal text generation completely non-functional.

□ The Mechanics & Equations

In classical older AI models (like Recurrent Neural Networks), reading a sentence order was built directly into the physical architecture. Words were read one at a time, strictly left-to-right.

$$\mathbf{h}_t = \sigma(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$$

In this formula, we are calculating the current "thought state" or hidden memory of the AI, represented by $\mathbf{h}_t$ (the t stands for the current time step). To get this current thought, we take the previous thought from the prior word, $\mathbf{h}_{t-1}$, and multiply it by a grid of adjustable dials called the weight matrix W_h . Then we take the current incoming word, $\mathbf{x}_t$, and multiply it by its own dial grid W_x . We add a baseline number $\mathbf{b}$ (a bias shift), and pass the whole thing through a squishing function σ to keep the numbers from exploding. Because step 3 mathematically cannot happen until step 2 is finished, the sequential order of words is permanently baked in.

In contrast, the Transformer throws away this step-by-step reading. It looks at all the words in a paragraph simultaneously using an attention mechanism.

$$\text{Attn}(X) = \text{softmax}\left(\frac{(XW_Q)(XW_K)^\top}{\sqrt{d_k}}\right)(XW_V)$$

Here, our entire paragraph of text is represented by a single giant grid of numbers, X . Every row in X is a word. We multiply the text grid by a set of learned dials W_Q to create "Query" questions, and by W_K to create "Key" answers. We multiply the Queries and the flipped (transposed, denoted by the tiny $\top$) Keys together to see which words match with each other. We divide by a stabilizing number, the square root of the dimension size ($\sqrt{d_k}$), to keep the math healthy. The softmax function simply turns these matching scores into percentages that add up to 100%. Finally, we multiply these percentages by the "Values" (our text multiplied by another dial grid W_V). There is no time here; it is a massive, instantaneous matching game.

Because everything happens at once, we must ask: what happens if we scramble the order of the words in the sentence? Let's define a shuffling grid, $\mathbf{P}$. This is a special matrix that simply swaps rows around.

$$\tilde{X} = \mathbf{P}X$$

The squiggly line over the X means it is the shuffled version of our text. We calculate this simply by multiplying our shuffling grid $\mathbf{P}$ against our original word grid X .

Now, let's see what happens to the Queries when the input text is shuffled.

$$\tilde{Q} = \mathbf{P}XW_Q = \mathbf{P}Q$$

The new shuffled Queries (squiggly $\tilde{Q}$) turn out to be exactly equal to taking the original Queries (Q) and simply shuffling their rows using the same exact shuffler $\mathbf{P}$. The exact same thing happens to the Keys and the Values.

Now we compute the attention matching scores with these shuffled inputs.

$$\tilde{A} = \text{softmax}\left(\frac{\mathbf{P}QK^\top \mathbf{P}^\top}{\sqrt{d_k}}\right) = \mathbf{P}A\mathbf{P}^\top$$

The new shuffled attention grid (squiggly $\tilde{A}$) is just the original attention grid A , but with its rows shuffled by $\mathbf{P}$ and its columns shuffled by the flipped version of the shuffler ($\mathbf{P}^\top$).

Finally, let's look at the final output of the entire attention mechanism when given the shuffled words.

$$\text{Attn}(\tilde{X}) = (\mathbf{P}A\mathbf{P}^\top)(\mathbf{P}V) = \mathbf{P}(AV) = \mathbf{P}\text{Attn}(X)$$

When we multiply the shuffled attention grid by the shuffled Values, the column-shuffle of the attention perfectly cancels out the row-shuffle of the Values. We are left with exactly the original output, just with its rows shuffled by $\mathbf{P}$. The actual mathematical scores *between* the words did not change by even a single decimal point. This proves mathematically that the Transformer is "permutation invariant." It cannot tell the difference between words that are side-by-side and words that are miles apart; shuffling the input simply shuffles the output without changing the calculations.

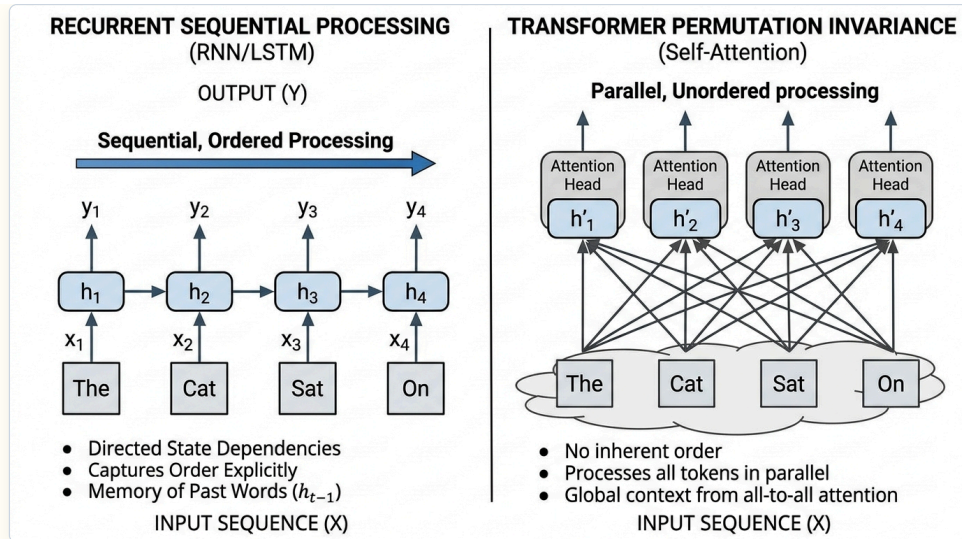


Figure 1.1: Recurrent Sequential Processing vs Transformer Permutation Invariance

⚙ Explicit Differentiation: Training vs. Inference

- **During Training (Learning the Weights):** When we are teaching the AI on massive GPU clusters, we feed in entire chunks of 4,096 words simultaneously. We process all 4,096 tokens in parallel matrix multiplications. Because raw attention is permutation-invariant, if we didn't artificially inject position tags, the model would simply treat the entire 4,096-token block as an unordered soup of concepts. It would learn what words generally appear together, but would completely fail to learn sentence structure, causality, or grammar.
- **During Inference / Generation (Runtime Chat with ChatGPT/Claude):** When a human user is chatting with the AI, the model predicts one single token at a time in a forward loop. Without native order awareness, it relies entirely on the position numbers stamped onto the newly generated token to differentiate it from the hundreds of previous tokens sitting in the "KV-Cache" (the model's short-term memory).

□ Concrete Numerical Toy Walk-through

Let's prove this with basic high-school arithmetic. Imagine our sentence is just two words: "Dog" and "Barks".

Let "Dog" be represented by the number 10. Let "Barks" be represented by the number 20.

Our normal, un-shuffled word grid X is:

Row 1: [10] (Dog)

Row 2: [20] (Barks)

Assume our simplified attention mechanism simply multiplies every row by every row.

For the Dog row (Row 1): it multiplies itself ($10 \times 10 = 100$) and it multiplies Barks ($10 \times 20 = 200$). Adding them together gives an output of **300**.

For the Barks row (Row 2): it multiplies Dog ($20 \times 10 = 200$) and it multiplies itself ($20 \times 20 = 400$). Adding them together gives an output of **600**.

Our final output list is **[300, 600]**.

Now, let's shuffle the input!

Our shuffled word grid $\tilde{X}$ is:

Row 1: [20] (Barks)

Row 2: [10] (Dog)

Let's run the exact same math.

For the new Row 1 (Barks): $20 \times 20 = 400$, plus $20 \times 10 = 200$. Total is **600**.

For the new Row 2 (Dog): $10 \times 20 = 200$, plus $10 \times 10 = 100$. Total is **300**.

Our final output list is **[600, 300]**.

Look at the result: the outputs are identical, just swapped in order. The network did the exact same math and arrived at the exact same numbers. It has absolutely no idea that "Dog" came first in the first example, and "Barks" came first in the second.

🌐 REAL-WORLD & NLP BEHAVIORS

- **Agent-Patient Inversion in Semantic Parsing:** Consider the sentences *"The cat hunted the mouse"* versus *"The mouse hunted the cat"*. Both sentences contain the identical bag of words: {"The", "cat", "hunted", "the", "mouse"}. Without an explicit positional signal, the unpermuted attention scores between "hunted" and "cat" are mathematically identical in both cases. The model cannot identify whether "cat" is the agent (predator) or the patient (prey), causing catastrophic failures in question answering and semantic role labeling.
- **Assignment Logic in Source Code Generation:** In programming languages, order dictates dataflow. The statement $x = y$ assigns the value of variable y into x , whereas $y = x$ assigns x into y . Without positional awareness, an attention layer computes identical self-attention affinities between the variable names and the assignment operator $=$ regardless of token position, making it impossible for a code-generation LLM to maintain variable scope or write syntactically correct software.

1.2 Absolute Sinusoidal Positional Encoding

💡 MOTIVATION & FUNCTIONAL ROLE

- **Importance:** Because Transformers are completely blind to word order, the original 2017 creators (Vaswani et al.) invented "absolute sinusoidal positional encodings." This injects a deterministic mathematical fingerprint of sequential order directly into the AI, breaking the permutation invariance without adding heavy learned parameters.
- **Functional Role:** It adds a unique, continuous, high-dimensional coordinate number array to each word's vector before the first layer. This encodes the absolute sequence position (e.g., "I am word number 5") while simultaneously allowing the math to calculate relative distances (e.g., "Word number 10 is exactly 5 steps away").
- **LLM Behavior Affected:** This allows the AI to attend to words based on where they physically sit in the paragraph (like attending to the immediate previous token, or the first token of a sentence). If implemented poorly, the model loses the ability to track long distances and produces gibberish when text gets too long.

📖 The Mechanics & Equations

The creators recognized they needed a positional fingerprint that was unique for every position, mathematically consistent regarding distance, and capable of handling paragraphs longer than anything the AI saw in training. They decided to use the natural rhythmic math of sine and cosine waves.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

This is the formula for the even-numbered slots in our word's vector array. We calculate a position encoding (PE). The pos variable represents the literal integer position of the word in the sentence (word 1, word 2, etc.). The $2i$ represents the even-numbered dimension slots in the vector (slot 0, slot 2, slot 4, etc.). The d_{model} is simply the total length of our word vector (usually hundreds of numbers long). We take the sine of this calculation. The massive number 10,000 acts as a stretching factor to pull the wave out wide.

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For the odd-numbered slots ($2i + 1$), we use the exact same formula but apply the cosine wave instead of the sine wave. By pairing sine and cosine together, we create a mathematical 2D circle—a rotating gear—for every pair of numbers in our word vector.

$$\omega_i = \frac{1}{10000^{2i/d_{model}}}$$

Let's extract the bottom half of that fraction and call it ω_i (omega). This represents the "angular frequency," which is a fancy term for how fast the wave ripples up and down. As we move deeper into the vector slots (as our index i gets larger), the frequency number gets extremely small, meaning the wave stretches out to become huge. The first few numbers in our vector ripple frantically with every single word step, acting like the minute hand on a clock. The last few numbers in the vector barely move over the span of a whole paragraph, acting like the hour hand.

Why go through all this trouble? Because sines and cosines have a magical mathematical property: they allow us to calculate relative distance.

$$\sin(\omega_i(pos + k)) = \sin(\omega_i pos) \cos(\omega_i k) + \cos(\omega_i pos) \sin(\omega_i k)$$

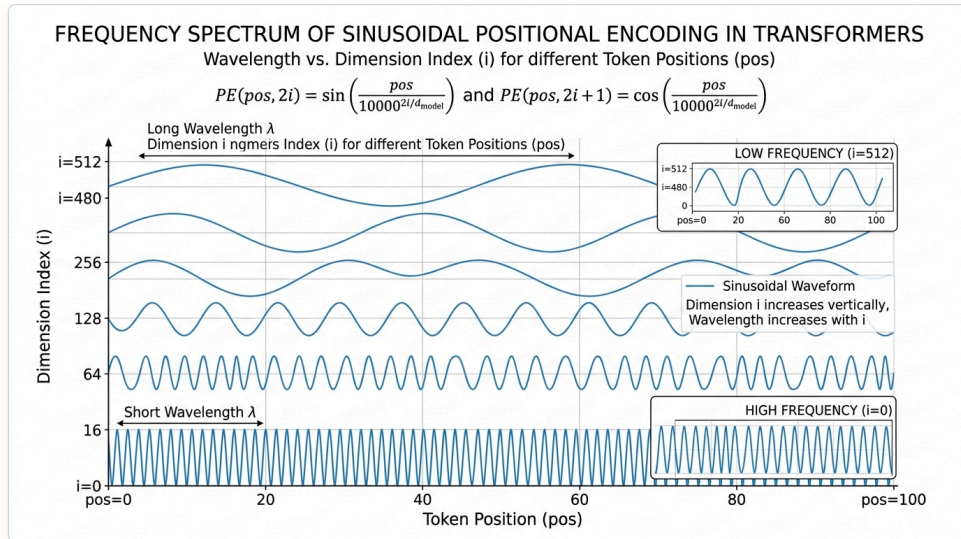
If we want to find the positional fingerprint of a word that is k steps away from our current position pos , standard high-school trigonometry tells us we can break the sine wave apart. The sine of the new shifted position ($pos + k$) is perfectly equal to a mix of the sine and cosine of the old position (pos) multiplied by the sine and cosine of the jump distance (k).

$$\cos(\omega_i(pos + k)) = \cos(\omega_i pos) \cos(\omega_i k) - \sin(\omega_i pos) \sin(\omega_i k)$$

The exact same rule applies to the cosine half. The new cosine for the shifted position is calculated strictly using the old position's sine and cosine and the jump distance k . This means the AI can always calculate a relative jump without needing to memorize absolute positions.

$$\begin{bmatrix} PE_{(pos+k, 2i)} \\ PE_{(pos+k, 2i+1)} \end{bmatrix} = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} PE_{(pos, 2i)} \\ PE_{(pos, 2i+1)} \end{bmatrix}$$

By organizing these sine and cosine rules into a 2-by-2 grid, we reveal the final secret: moving k steps forward in the text is mathematically identical to spinning our position vector like a dial on a safe. A fixed jump distance k always results in the exact same rotation grid, no matter what absolute position pos we started from. The AI learns to turn these dials to precisely locate words at specific distances.



🌀 Explicit Differentiation: Training vs. Inference

- **During Training (Learning the Weights):** When pushing massive 4,096-token texts through the GPU clusters, the system doesn't calculate sines and cosines one by one. It creates a giant, pre-calculated sheet of sine and cosine waves covering positions 1 through 4,096. It literally adds this massive sheet of mathematical static on top of the entire block of word vectors in a single blindingly fast addition operation before the data ever touches the first layer of the network.
- **During Inference / Generation (Runtime Chat with ChatGPT/Claude):** When the AI is generating the 100th word in a response, the system computes the sine and cosine fingerprint uniquely for position 100, and adds it to the 100th word's vector. The AI compares this new token to the vectors for positions 1 to 99 sitting in the KV-Cache, which already had their specific position waves permanently baked into them during their own earlier forward passes.

🧮 Concrete Numerical Toy Walk-through

Let's see how this works with simple arithmetic. Let our word vector be extremely tiny, just 2 numbers long. Let's pretend our frequency stretching factor ω_i is simply 1 for easy math.

For the word at **Position 1**:

The even slot: $\sin(1 \times 1) = \sin(1) \approx 0.84$

The odd slot: $\cos(1 \times 1) = \cos(1) \approx 0.54$

So the permanent positional fingerprint tag for Position 1 is **[0.84, 0.54]**.

For the word at **Position 2**:

The even slot: $\sin(1 \times 2) = \sin(2) \approx 0.91$

The odd slot: $\cos(1 \times 2) = \cos(2) \approx -0.42$

So the permanent positional fingerprint tag for Position 2 is **[0.91, -0.42]**.

The AI literally takes the underlying meaning vector for the first word and adds [0.84, 0.54] to it. It takes the meaning vector for the second word and adds [0.91, -0.42] to it. Every single position on the page gets a wholly unique numerical fingerprint added directly into the word's data.

🌐 REAL-WORLD & NLP BEHAVIORS

- **Local Syntactic Agreement vs. Global Discourse Tracking:** In German or Russian, case markers and adjective-noun agreements occur within short distances (1–3 tokens), whereas verb clauses in subordinate sentences can be separated by dozens of tokens. Sinusoidal encoding enables the model to allocate high-frequency dimensions (the fast rippling minute hands) to track local grammatical dependencies while low-frequency dimensions (the slow hour hands) preserve absolute paragraph-level narrative pacing.
- **Failure in Length Extrapolation (The Absolute Position Barrier):** When early models trained with absolute sinusoidal encodings were evaluated on paragraphs longer than their training window (e.g., reading 4096 words after only training on 2048 words), performance collapsed completely. The sines and cosines at positions beyond 2048 generated numbers the network had never practiced with, causing the mathematical scores to explode. This proved that simply *adding* absolute coordinates directly into token embeddings pollutes semantic representations at scale.

1.3 Rotary Position Embedding (RoPE)

🔗 MOTIVATION & FUNCTIONAL ROLE

- **Importance:** Rotary Position Embedding (RoPE) is the modern standard used across almost all cutting-edge AI models today (LLaMA 1/2/3, Mistral, Qwen, DeepSeek). It fixes the core flaw of the older sinusoidal method by applying position through pure rotation instead of messy addition.
- **Functional Role:** Instead of adding position vectors to the original word vectors, RoPE steps inside the attention mechanism and physically rotates the Query and Key vectors in geometric space. This guarantees that when two words match, their score depends *strictly* on their relative distance from each other, leaving their underlying word meanings completely unpolluted.
- **LLM Behavior Affected:** This rotation trick allows models to flawlessly read monstrously large documents (up to 128,000 words or more). It naturally causes the attention span to slowly decay over long distances (reducing noise), and allows engineers to instantly double or quadruple a model's reading capacity simply by slowing down the rotational gears.

📐 The Mechanics & Equations

In the older absolute method we just discussed, adding position tags $\mathbf{p}_m$ and $\mathbf{p}_n$ to the input word vectors creates a massive mathematical mess.

$$\begin{aligned}\mathbf{q}_m^\top \mathbf{k}_n &= (\mathbf{x}_m + \mathbf{p}_m) W_Q W_K^\top (\mathbf{x}_n + \mathbf{p}_n)^\top \\ &= \mathbf{x}_m W_Q W_K^\top \mathbf{x}_n^\top + \mathbf{x}_m W_Q W_K^\top \mathbf{p}_n^\top \\ &\quad + \mathbf{p}_m W_Q W_K^\top \mathbf{x}_n^\top + \mathbf{p}_m W_Q W_K^\top \mathbf{p}_n^\top\end{aligned}$$

Here, we are calculating the matching score between a Query word at position m (with meaning $\mathbf{x}_m$ and position $\mathbf{p}_m$) and a Key word at position n (with meaning $\mathbf{x}_n$ and position $\mathbf{p}_n$). They are multiplied by their dial grids W_Q and W_K . Because of basic high-school algebra (the FOIL method for multiplying terms in parentheses), the math violently expands into four entangled pieces: word-to-word matching, word-to-position matching, position-to-word matching, and position-to-position matching. The AI is forced to waste massive amounts of brainpower trying to untangle the actual word meanings from the noisy coordinate numbers.

We need a better way. The researchers who invented RoPE set a strict mathematical goal.

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = g(\mathbf{x}_m, \mathbf{x}_n, m - n)$$

They wanted a magic function for the Query (let's call it f_q) and a magic function for the Key (f_k) so that when you multiply them together (denoted by the angle brackets $\langle \rangle$, representing a dot-product), the final matching score g depends *exclusively* on the word contents ($\mathbf{x}_m, \mathbf{x}_n$) and the pure relative distance between them ($m - n$). No addition, no messy cross-contamination.

They found the solution in the geometry of circles using Euler's formula.

$$R_{\theta, m} z = (x_1 + ix_2) e^{im\theta}$$

If we take any pair of numbers from our word vector (x_1 and x_2) and pretend they are a single coordinate point on a 2D map, we can write it as a complex number ($z = x_1 + ix_2$). To rotate this point around the center of the map by an angle based on the word's position m and a base frequency θ , we simply multiply it by the exponential rotation formula $e^{im\theta}$.

$$(x_1 + ix_2) e^{im\theta} = (x_1 \cos(m\theta) - x_2 \sin(m\theta)) + i(x_1 \sin(m\theta) + x_2 \cos(m\theta))$$

When we expand that complex rotation math, it yields a simple formula that gracefully mixes the original two numbers (x_1 and x_2) using sines and cosines of the combined angle $m\theta$.

$$R_{\theta, m} = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

We can pluck those sines and cosines out into a neat 2-by-2 rotation grid, called $R_{\theta, m}$. If you multiply any pair of numbers by this 2-by-2 grid, it perfectly rotates them by the angle $m\theta$ without changing their length.

$$\mathbf{R}_{\Theta, m}^d = \text{diag}(R_{\theta_1, m}, R_{\theta_2, m}, \dots, R_{\theta_{d/2}, m})$$

Because our actual word vectors have hundreds of dimensions (d), we build a giant diagonal grid (denoted by the bold $\mathbf{R}$) made of many little 2-by-2 rotation grids stacked diagonally corner-to-corner. Each little 2-by-2 block gets its own unique frequency (θ_1, θ_2 , etc.), meaning every pair of dimensions in the word vector spins at a completely different speed as we move through the text.

$$\langle \mathbf{R}_{\Theta, m}^d \mathbf{q}_m, \mathbf{R}_{\Theta, n}^d \mathbf{k}_n \rangle = (\mathbf{R}_{\Theta, m}^d \mathbf{q}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{k}_n)$$

Now, we actually apply this rotation to the attention mechanism. We take our pure Query vector $\mathbf{q}_m$ and rotate it by the giant grid for position m . We take our pure Key vector $\mathbf{k}_n$ and rotate it by the giant grid for position n . Then we multiply them together to get the attention score. Notice that we applied rotation *after* making the Query and Key. There is no messy addition happening here.

$$(\mathbf{R}_{\Theta, m}^d)^\top \mathbf{R}_{\Theta, n}^d = \mathbf{R}_{\Theta, -m}^d \mathbf{R}_{\Theta, n}^d = \mathbf{R}_{\Theta, n-m}^d$$

The incredible beauty of rotation grids is that if you "un-rotate" by m steps (the flipped grid $(\mathbf{R}_{\Theta, m}^d)^\top$, which equals rotating by $-m$) and then "rotate" by n steps, the math perfectly collapses into a single rotation of exactly $n - m$ steps.

$$\langle \mathbf{R}_{\Theta, m}^d \mathbf{q}_m, \mathbf{R}_{\Theta, n}^d \mathbf{k}_n \rangle = \mathbf{q}_m^\top \mathbf{R}_{\Theta, n-m}^d \mathbf{k}_n$$

And there is the magic. The final attention score between the two words is mathematically perfectly equal to taking the raw, unpolluted Query and Key vectors and rotating them strictly by the relative distance between them ($n - m$). Absolute positions completely vanish from the result.

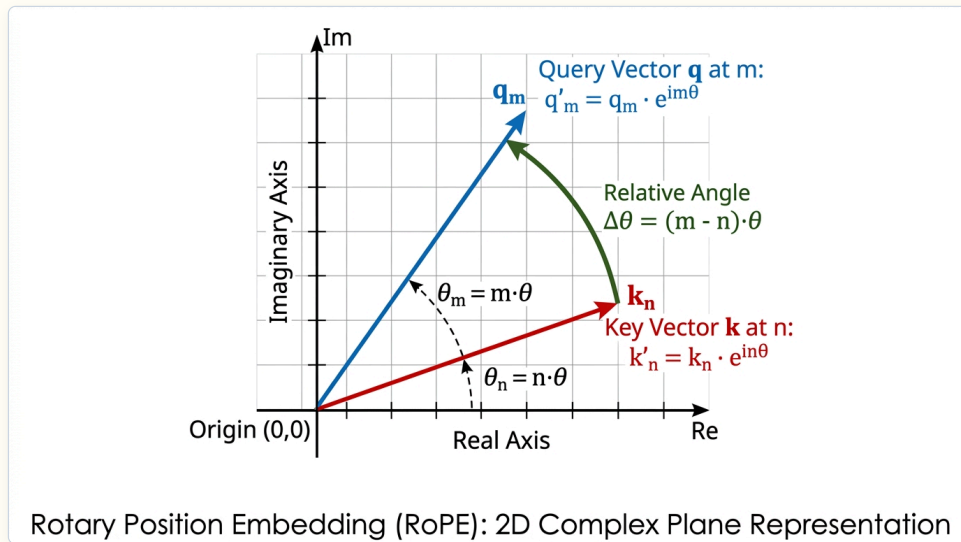


Figure 1.3: Rotary Position Embedding 2D Complex Subspace Rotation

⚙ Explicit Differentiation: Training vs. Inference

- **During Training (Learning the Weights):** We process giant batches of thousands of tokens simultaneously. The network computes all the raw Queries and Keys for the entire sequence at once. Then, a massive pre-computed mathematical tensor of rotation matrices (representing every single position angle) is multiplied against the Queries and Keys in one giant parallel sweep. This guarantees every single matching dot-product in the whole paragraph instantly reflects pure relative distance.
- **During Inference / Generation (Runtime Chat with ChatGPT/Claude):** When generating a new word at position 100, the system creates the Query for position 100 and physically rotates it by angle 100. It computes the Key for position 100, rotates it by angle 100, and stores it in the KV-Cache short-term memory. When it needs to check its attention against an old word at position 90, it simply multiplies Query-100 by the already-cached Key-90. The relative rotation math natively handles the 10-step distance automatically without any extra work.

□ Concrete Numerical Toy Walk-through

Imagine our Query vector for the word "Cat" at position 5 is simply $[1, 0]$.

Imagine our Key vector for the word "Sat" at position 2 is also $[1, 0]$.

Let our rotation speed be exactly 90 degrees per position step.

Let's rotate the Query by its position (5): $5 \text{ steps} \times 90 \text{ degrees} = 450 \text{ degrees}$ (which wraps around to 90 degrees). Rotating the point $[1, 0]$ by 90 degrees points it straight up: it becomes $[0, 1]$.

Let's rotate the Key by its position (2): $2 \text{ steps} \times 90 \text{ degrees} = 180 \text{ degrees}$. Rotating the point $[1, 0]$ by 180 degrees points it backwards: it becomes $[-1, 0]$.

Now we multiply the rotated Query and the rotated Key together (multiplying matching slots and adding):

$$(0 \times -1) + (1 \times 0) = 0 + 0 = 0.$$

Now, let's look at the relative math. The distance between position 5 and position 2 is exactly 3 steps.

$$3 \text{ steps} \times 90 \text{ degrees} = 270 \text{ degrees}.$$

If we took our raw unpolluted vectors $[1, 0]$ and $[1, 0]$, and just rotated the second one by the relative 270 degrees, it points straight down: $[0, -1]$.

If we multiply the first vector $[1, 0]$ against this relative vector $[0, -1]$:

$$(1 \times 0) + (0 \times -1) = 0 + 0 = 0.$$

The math proves out exactly! The absolute positions 5 and 2 didn't matter at all; only the pure relative distance of 3 steps completely controlled the final interaction score.

🌐 REAL-WORLD & NLP BEHAVIORS

- **Needle In A Haystack (NIAH) Long-Context Retrieval:** In long-context tests (like hiding a secret sentence inside a 128,000-word book), models using the older addition method suffer massive failure because the huge position numbers create screaming mathematical noise that drowns out the words. Because RoPE rotates instead of adds, it naturally quiets down the attention scores as words get extremely far apart (long-term decay), while perfectly preserving the word meaning, allowing models like LLaMA-3 to achieve 100% retrieval accuracy.
- **Context Extension via RoPE Frequency Scaling:** When engineers want to force a model trained on 4,000 words to suddenly read 128,000 words, they don't have to retrain the whole model. They simply alter the base frequencies θ_i of the RoPE gears (slowing them down). Because RoPE's geometry is based on continuous rotation, the network simply interprets the massive 128,000 word book as if it were a highly-compressed 4,000 word book, immediately granting the AI an expanded reading capacity with almost zero extra cost.

Module 2: Self-Attention Dissected (The Core Router)

Self-attention is the fundamental routing primitive of the Transformer architecture (Vaswani et al., 2017). Unlike static recurrent u

2.1 The Three Projections

🤔 Why?

- **Decoupled Functional Roles**: In natural language, a token's identity as a searcher (what it is looking for), as an indexable tar
- **Architectural Role**: The projection matrices linearly project raw token representations into three specialized representational
- **Failure Mode Without Projections**: If an architecture computes attention directly on raw embeddings, the affinity matrix is symm

🏗️ Definition & Architecture

Vaswani et al. (2017) formalized attention as mapping a query and a set of key-value pairs to an output. Given an input sequence matr

![[Figure 2.1: Query, Key, and Value Linear Projections]](assets/diagram_2_1_projections.jpg)

xml

Input Matrix $X [N \times d_{\text{model}}]$

$W_Q [d_{\text{model}} \times d_k]$

$W_K [d_{\text{model}} \times d_k]$

$W_V [d_{\text{model}} \times d_v]$

Queries $Q = X \cdot W_Q$

Keys $K = X \cdot W_K$

Values $V = X \cdot W_V$

□ The Mathematics & Working

To begin the self-attention process, we must first determine what each token in our sequence is looking for. This is its search profile.

$$Q = X W_Q$$

In this equation, X represents our input sequence matrix, which holds the raw numerical vectors for every token in our sentence. We

Toy Numerical Walk-Through for Queries:

Imagine our input X is a single token vector represented by $[2, 1]$. Our learned query weight matrix W_Q is a simple 2×2

But knowing what we are searching for is not enough; each token must also advertise what information it contains so others can find it.

$$K = X W_K$$

Here, we take the exact same input token matrix X and multiply it by a completely different learned weight matrix called W_K . This

Toy Numerical Walk-Through for Keys:

Let's reuse our input token vector $X = [2, 1]$. Now, suppose our key weight matrix W_K has row one as $[-1, 0]$ and row two as $[0$

Finally, if a token is successfully searched for and found, it needs to have a concrete message or payload to send back.

$$V = X W_V$$

We again start with the input matrix X and multiply it by a third distinct set of learned weights, W_V . This calculation yields the

Toy Numerical Walk-Through for Values:

Using the same $X = [2, 1]$, let our value weight matrix W_V have row one as $[0, 1]$ and row two as $[1, 0]$. Multiplying X by W_V

2-Sentence Plain Lingo:

The three projections transform each token vector into three separate roles: an inquiry asking what information is needed, an address

⊗ Training vs. Inference

- **During Training (Learning the Weights):**

The model processes massive blocks of text simultaneously. The input X contains thousands of tokens processed in parallel across

- **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**

The matrices W_Q , W_K , and W_V are completely frozen. As you type a prompt, the model converts your latest word into a vector

□ Real-World & NLP Behaviors (2 Examples)

- **Example 1: Asymmetric Subject-Verb Routing:** In "The chef who prepared the dishes was exhausted", the singular verb "was" mu

- **Example 2: Anaphora / Coreference Resolution:** In "The trophy didn't fit into the brown suitcase because it was too large", th

2.2 Raw Compatibility Scoring

□ Why?

- **Dynamic Content-Based Routing:** To route information adaptively across a sequence, the model must measure pairwise relevance bet

- **Architectural Role:** The raw compatibility scoring calculates the unscaled dot-product similarity (affinity logits) between ever

- **Failure Mode Without Dot-Product Affinity:** Without an inner-product compatibility metric, the network would require either feed

□ Definition & Architecture

In inner-product spaces, the dot product between two vectors measures their alignment. When a query vector is multiplied by a key vec

![Figure 2.2: Raw Compatibility Scoring Matrix Multiplication](assets/diagram_2_2_dot_product.jpg)

□ The Mathematics & Working

To find out how well a single word's search query matches another word's identity tag, we calculate their direct mathematical similar

$$s_{i,j} = \sum_{m=1}^{d_k} q_{i,m} k_{j,m}$$

In this formula, $s_{i,j}$ is the raw compatibility score between token i and token j . We take the query vector of the searching

Toy Numerical Walk-Through for a Single Match:

Let our query vector q_1 be $[2, 1]$ and our key vector k_2 be $[-2, 2]$. We multiply the first components: $2 \times -2 = -4$. We

However, calculating this one by one for every pair of words in a sentence is incredibly slow. We need a way to score every single qu

$$S_{\text{raw}} = Q K^T$$

Here, S_{raw} represents a massive grid (a matrix) containing the scores of every token paired against every other token. We

****Toy Numerical Walk-Through for the Full Matrix:****

Assume we have two tokens. Our query matrix Q is $\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$ (row 1 is query 1, row 2 is query 2). Our key matrix K is $\begin{bmatrix} -2 & 2 \\ 3 & 1 \end{bmatrix}$. Now we multiply $Q \times K^T$:

- Score of Query 1 vs Key 1: $2 \times -2 + 1 \times 3 = -1$
- Score of Query 1 vs Key 2: $2 \times 2 + 1 \times 1 = 5$
- Score of Query 2 vs Key 1: $0 \times -2 + 1 \times 3 = 3$
- Score of Query 2 vs Key 2: $0 \times 2 + 1 \times 1 = 1$

Our resulting S_{raw} matrix is $\begin{bmatrix} -1 & 5 \\ 3 & 1 \end{bmatrix}$.

****2-Sentence Plain Lingo**:**

The model calculates the affinity between every pair of words by multiplying each word's query vector against every other word's key

⊗ Training vs. Inference

- ****During Training (Learning the Weights):****
The model computes this massive grid S_{raw} for enormous chunks of text at once (e.g., 4,096 tokens). To prevent the mode
- ****During Inference / Generation (Runtime Chat with ChatGPT/Claude):****
When you are generating the next word in a chat, the model only has one new query vector for the current step. It multiplies this s

□ Real-World & NLP Behaviors (2 Examples)

- ****Example 1: Prepositional Phrase Attachment**:** In the sentence **"She saw the man with a telescope"**, the phrase **"with a telescope**
- ****Example 2: Polysemy Disambiguation**:** For the ambiguous word **"bank"** in **"He deposited money along the river bank"**, the query f

2.3 Variance Scaling Factor

□ Why?

- ****Preventing Gradient Vanishing**:** As the projection dimension grows large, the dot product between random independent zero-mean un
- ****Architectural Role**:** Dividing the raw logits by the square root of the dimension stabilizes the variance of the dot products bac
- ****Failure Mode Without Scaling**:** Without this scaling factor, for typical LLM dimensions (like 128), dot products easily reach mag

□ Definition & Architecture

Vaswani et al. (2017) observed that while dot-product attention is highly space-efficient and fast, it underperforms for large vector

![[Figure 2.3: Variance Scaling Factor and Gradient Active Region](assets/diagram_2_3_scaling.jpg)]

□ The Mathematics & Working

To fix the exploding variance problem, we must scale down every single score in our raw compatibility matrix.

$$s_{i,j} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

In this equation, we take the raw compatibility score (the dot product $q_i \cdot k_j$) we computed previously and divide it by the s

****Toy Numerical Walk-Through for Single Score Scaling:****

Imagine our vector length d_k is 4. The square root of 4 is 2. If our raw dot product score between a query and a key was inc

However, just as before, we apply this operation to the entire grid of scores at once.

$$S = \frac{Q K^T}{\sqrt{d_k}}$$

Here, S is the final, scaled affinity matrix. We take the entire raw matrix grid $Q K^T$ that we calculated in the previous step an

****Toy Numerical Walk-Through for Matrix Scaling:****

Let's reuse our S_{raw} matrix from the previous section: $\begin{bmatrix} -1 & 5 \\ 3 & 1 \end{bmatrix}$. Assume our vector length d_k was 4, so ou

- Top left: $-1 / 2 = -0.5$
- Top right: $5 / 2 = 2.5$
- Bottom left: $3 / 2 = 1.5$
- Bottom right: $1 / 2 = 0.5$

Our new scaled matrix S is $\begin{bmatrix} -1 & 1.5 \\ 1 & 0.5 \end{bmatrix}$. The relationships are identical, but the absolute numbers are tamed.

****2-Sentence Plain Lingo**:**

Multiplying high-dimensional vectors adds up dozens of numbers, causing the total score to blow up to extreme values that would freeze

☉ Training vs. Inference

- ****During Training (Learning the Weights):****

This scaling step is absolutely critical. Without it, the very first batch of training data would generate massive numbers, causing

- ****During Inference / Generation (Runtime Chat with ChatGPT/Claude):****

The scaling factor acts as a static temperature control. Because d_k is a fixed architectural constant (e.g., 128 for many models

☐ Real-World & NLP Behaviors (2 Examples)

- ****Example 1: Initial Training Convergence of Large LLMs**:** In early pretraining iterations of models like LLaMA-3 (where vector len

- ****Example 2: Multi-Token Soft Focus in Machine Translation**:** When translating an idiomatic phrase like "kick the bucket" into an

2.4 Softmax Normalization

☐ Why?

- ****Probabilistic Convex Combination**:** Raw scaled affinity scores are unbounded real values (ranging from negative infinity to posit

- ****Architectural Role**:** Applying row-wise softmax normalizes the affinity scores such that all scores sum exactly to 1.0 , and no

- ****Failure Mode Without Softmax**:** If unnormalized scores were used, sequence outputs would fluctuate wildly in magnitude depending

☐ Definition & Architecture

The softmax function takes a list of numbers, exponentiates each one (raises the mathematical constant e to the power of that numbe

For numerical stability in hardware implementations, modern runtime kernels utilize mathematically safe versions of softmax to avoid

![[Figure 2.4: Softmax Row-wise Probability Normalization](assets/diagram_2_4_softmax.jpg)]

☐ The Mathematics & Working

We need to convert our unbounded scaled scores into strict percentage-based probabilities.

$$A_{i,j} = \frac{\exp(s_{i,j})}{\sum_{l=1}^N \exp(s_{i,l})}$$

In this formula, $A_{i,j}$ represents the final attention weight: the exact percentage of attention that token i will pay to token

****Toy Numerical Walk-Through for Softmax:****

Let's look at Query 1's scaled scores from our previous matrix: $\begin{bmatrix} -1 & 1.5 \end{bmatrix}$. Token 1 scored a -1 for Token 1, and a 1.5 for Token 2. First, we exponentiate them:

- $\exp(-1) \approx 0.368$

- $\exp(1.5) \approx 4.482$

Next, we sum them up: $0.368 + 4.482 = 4.85$.

Finally, we divide each by the sum to get the percentages:

- For Token 1: $0.368 / 4.85 \approx 0.076$ (or 7.6%)

- For Token 2: $4.482 / 4.85 \approx 0.924$ (or 92.4%)

Token 1 will give 92.4% of its attention to Token 2, and only 7.6% to itself. The total sum is exactly 100% .

To express this process happening to the entire grid at once, we write it as a matrix operation.

$$A = \text{softmax}(S)$$

Here, A is the final Attention weight matrix. The function $\text{softmax}()$ applies the exponentiation and division across every

****Toy Numerical Walk-Through for Matrix Softmax:****

Using our full scaled matrix $S = \begin{bmatrix} -1 & 1.5 \\ 1 & 0.5 \end{bmatrix}$.

Row 1 is $\begin{bmatrix} -1 & 1.5 \end{bmatrix}$. As calculated above, it becomes $\begin{bmatrix} 0.076 & 0.924 \end{bmatrix}$.

Row 2 is $\begin{bmatrix} 1 & 0.5 \end{bmatrix}$. Let's exponentiate: $\exp(1) \approx 2.718$, $\exp(0.5) \approx 1.649$. The sum is 4.367 .

- Token 2 to Token 1: $2.718 / 4.367 \approx 0.622$ (62.2%)

- Token 2 to Token 2: $1.649 / 4.367 \approx 0.378$ (37.8%)

Our final Attention matrix A is $\begin{bmatrix} 0.076 & 0.924 \\ 0.622 & 0.378 \end{bmatrix}$. Every row sums to exactly 1.0 .

****2-Sentence Plain Lingo**:**

The softmax operation turns arbitrary compatibility numbers into a percentage-based budget that sums up to exactly one hundred percent

```

#### ☉ Training vs. Inference
- **During Training (Learning the Weights):**
    During training, the softmax calculation operates over massive matrices. Because of the causal mask applied earlier (replacing futu
- **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**
    Because the model generates one word at a time, it calculates the softmax over a continuously growing list of cached keys. As the c

#### ☐ Real-World & NLP Behaviors (2 Examples)
- **Example 1: Long-Context Distraction / Needle-in-a-Haystack**: In a 32,000-token context window, if many irrelevant tokens have sl
- **Example 2: Syntactic Tree Induction in Transformer Heads**: Probing studies show that specific attention heads use the softmax di

***

### 2.5 Weighted Value Aggregation

#### ☐ Why?
- **Synthesizing Context-Enriched Representations**: Attention scores alone are merely routing weights; they carry no semantic conten
- **Architectural Role**: The final stage takes a linear combination of all value vectors weighted by their attention probabilities t
- **Failure Mode Without Value Aggregation**: If an architecture directly returned the attention matrix or just summed queries and ke

#### ☐ Definition & Architecture
Vaswani et al. (2017) complete the scaled dot-product attention equation by performing a weighted sum of the value vectors. For any g

![[Figure 2.5: Weighted Value Aggregation and Output Representation]](assets/diagram_2_5_value_aggregation.jpg)

#### ☐ The Mathematics & Working

To create the final, updated understanding for a single token, we must mix the payloads of the tokens it paid attention to.


$$z_i = \sum_{j=1}^N A_{i,j} v_j$$


In this equation,  $z_i$  is the final output vector for token  $i$ . To build it, we take the attention percentage  $A_{i,j}$  (how much t

**Toy Numerical Walk-Through for a Single Token:**
Let's build the final output  $z_1$  for Query 1. From our attention matrix  $A$ , Token 1 gave 0.076 ($7.6\%$) attention to Token 1, a
Assume the Value vectors we calculated all the way back in section 2.1 are  $v_1 = [1, 2]$  and  $v_2 = [3, 0]$ .
We multiply each value vector by its percentage:
-  $0.076 \times [1, 2] = [0.076, 0.152]$ 
-  $0.924 \times [3, 0] = [2.772, 0]$ 
We add them together:  $[0.076 + 2.772, 0.152 + 0] = [2.848, 0.152]$ .
The new, context-updated vector  $z_1$  is  $[2.848, 0.152]$ .

To execute this mixing for every token in the sequence instantly, we use matrix multiplication.


$$Z = A V$$


Here,  $Z$  is the final output matrix containing the newly updated vectors for every token in our sequence. We take the complete atten

**Toy Numerical Walk-Through for the Full Matrix:**
Our full attention matrix  $A$  is  $\begin{bmatrix} 0.076 & 0.924 \\ 0.622 & 0.378 \end{bmatrix}$ .
Our value matrix  $V$  is  $\begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix}$ .
Multiplying  $A \times V$ :
- Row 1 (Token 1 output): as calculated above,  $[2.848, 0.152]$ 
- Row 2 (Token 2 output):  $(0.622 \times 1) + (0.378 \times 3) = 1.756$ . And  $(0.622 \times 2) + (0.378 \times 0) = 1.244$ . Vector is
Our final output matrix  $Z$  is  $\begin{bmatrix} 2.848 & 0.152 \\ 1.756 & 1.244 \end{bmatrix}$ .

**2-Sentence Plain Lingo:**
The model uses the percentage scores from the softmax step to mix together the actual content payloads of every word in the sentence.

#### ☉ Training vs. Inference
- **During Training (Learning the Weights):**
    The operation  $Z = A V$  is computed as a dense, heavily optimized matrix multiplication across the entire block of text. The result
- **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**
    When generating a response, the model takes the newly computed attention percentages for the current single word and mixes them aga

#### ☐ Real-World & NLP Behaviors (2 Examples)
- **Example 1: Contextual Word Sense Specialization**: In "Apple released the new M4 MacBook Pro", the initial embedding for "Appl
- **Example 2: Cross-Lingual Feature Alignment**: In neural machine translation models, a target token in English attending across th

```

Module 3: Multi-Head & Causal Decoding

While a single attention calculation is great at routing information between words, a single "brain" (or head) has a major limitation

3.1 Multi-Head Subspace Splitting (\$h\$ Parallel Heads)

□ Why?

- **Multiple Simultaneous Interaction Channels**: When you read a sentence, your brain does several things at exactly the same time.
- **Architectural Role**: To solve this, we give the model multiple "heads" (represented by the variable \$h\$). The model shrinks its
- **Failure Mode Without Multi-Head Splitting**: If a model uses only one massive attention head, it loses precision. It cannot simul

□ Definition, Architecture, and The Mathematics

Instead of performing one massive attention calculation, we split the task. Let's look at the mathematical journey of a single head,

$$\text{head}_i = \text{Attention}(X W_i^Q, X W_i^K, X W_i^V)$$

In this first equation, we are setting up the inputs for one specific expert head. The matrix \$X\$ represents our incoming sequence of

Now that we have our reduced-size queries, keys, and values, the head performs its own isolated attention calculation:

$$\text{head}_i = \text{softmax}\left(\frac{(X W_i^Q)(X W_i^K)^T}{\sqrt{d_k}}\right) (X W_i^V)$$

Here, we see the inner workings of head \$i\$. First, it takes its specialized query \$(X W_i^Q)\$ and multiplies it by its specialized,

![[Figure 3.1: Multi-Head Attention Subspace Splitting](assets/diagram_3_1_multihead.jpg)]

Because we have many heads running at the same time, we need a way to bring all their isolated findings back together:

$$\text{MultiHead_Raw} = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)$$

Once every single head (from head \$1\$ all the way to head \$h\$) has completed its specific mathematical task, we use the `Concat` (con

□ Explicit Differentiation: Training vs. Inference

- **During Training (Learning the Weights)**:

On massive GPU clusters, the model processes thousands of documents simultaneously. The individual weight matrices (\$W_i^Q\$, \$W_i^K\$

- **During Inference / Generation (Runtime Chat with ChatGPT/Claude)**:

When you are chatting with the model, these weight matrices are entirely frozen. No learning is happening. As you stream text into

□ Concrete Numerical Toy Walk-Through

Let's pretend our model has a total word dimension of \$4\$, and we want to split it into \$h = 2\$ parallel heads.

Our input word vector \$X\$ for the word "cat" is \$[2, 4, 6, 8]\$.

To find the size for each head (\$d_k\$), we divide the total dimension by the number of heads: \$4 / 2 = 2\$.

- Head 1 has a learned query matrix \$W_1^Q\$ that mathematically extracts the first two numbers. So, the Query for Head 1 becomes \$[2, 4]\$.
- Head 2 has a query matrix \$W_2^Q\$ that extracts the last two numbers. The Query for Head 2 becomes \$[6, 8]\$.

Each head runs its isolated attention math. Let's imagine Head 1 outputs the final answer \$[1, -1]\$, and Head 2 outputs \$[0, 5]\$.

We apply the `Concat` operation to paste them side-by-side:

Final Raw Output = \$[1, -1, 0, 5]\$.

We started with 4 numbers, split them into specialized groups of 2, and cleanly pasted them back into a unified set of 4 numbers.

**2-Sentence Plain Lingo

Instead of having one single observer analyze the entire sentence, the model divides its thinking space into multiple parallel expert

□ Real-World & NLP Behaviors (2 Examples)

- **Example 1: Specialized Syntactic and Positional Induction**: In empirical attention analysis of GPT-class architectures, Head 1 i
- **Example 2: Multi-Hop Question Answering**: When answering "Where was the author of 'Hamlet' born?", one attention head binds "

3.2 Output Linear Projection (W^0)

Why?

- **Subspace Integration and Dimensional Re-alignment**: In the previous section, we glued all the head outputs side-by-side. However
- **Architectural Role**: We use an output projection matrix called W^0 . This matrix acts as a blending board, taking the glued-together
- **Failure Mode Without Output Projection**: If we just pasted the heads together without mixing them, the network would have disjoint

Definition, Architecture, and The Mathematics

First, let's formalize the pasting action from the previous step:

$$H_{\text{concat}} = [H_1 \parallel H_2 \parallel \dots \parallel H_h]$$

In this equation, H_{concat} represents our raw, concatenated matrix. The symbol $\parallel$ stands for concatenation (gluing). We

Now, we must mix these independent findings together:

$$Z_{\text{out}} = H_{\text{concat}} W^0$$

Here, we take our wide pasted matrix H_{concat} and multiply it by a brand-new, learned weight matrix called W^0 (the Output

! [Figure 3.2: Multi-Head Concatenation and Output Projection W_0] (assets/diagram_3_2_output_proj.jpg)

There is an alternative, mathematically identical way to think about this mixing process:

$$Z_{\text{out}} = \sum_{i=1}^h H_i W_i^0$$

Instead of pasting everything first, this equation shows that we can multiply each individual H_i by its own dedicated sl

Explicit Differentiation: Training vs. Inference

- **During Training (Learning the Weights)**:
Initially, the mixing board W^0 is full of random numbers. The model does not know how to reconcile conflicting information from
- **During Inference / Generation (Runtime Chat with ChatGPT/Claude)**:
During a live chat, W^0 is a fixed grid of numbers. For every single token you type, the model calculates the multiple heads, pas

Concrete Numerical Toy Walk-Through

Let's look at how the math actually blends numbers. From our previous step, our glued-together vector was $H_{\text{concat}} = [1, -1$
Imagine our mixing matrix W^0 has learned the following simple rules through training:

- **Output Rule 1**: Add the first and last numbers together.
- **Output Rule 2**: Subtract the second number from the third.
- **Output Rule 3**: Multiply the last number by 2.
- **Output Rule 4**: Just keep the first number.

Let's do the basic arithmetic:

- New Number 1: $1 + 5 = 6$
- New Number 2: $0 - (-1) = 1$
- New Number 3: $5 \times 2 = 10$
- New Number 4: 1

Our final mixed output Z_{out} is $[6, 1, 10, 1]$.

Notice what happened! Because of the mixing matrix W^0 , the independent numbers from Head 1 ($1, -1$) and Head 2 ($0, 5$) have now

2-Sentence Plain Lingo

Once all the separate attention heads finish their individual analyses, their answers are lined up side by side and passed through a

Real-World & NLP Behaviors (2 Examples)

- **Example 1: Cross-Head Feature Cancellation and Conflict Resolution**: When Head 2 flags a token as a potential noun and Head 5 fl
- **Example 2: Residual Stream Alignment in Instruction Tuning**: In residual-stream interpretability studies (e.g., Elhage et al., 2

3.3 Causal Triangular Attention Masking

Why?

- **Preserving Autoregressive Causality**: Language models like GPT generate text sequentially, from left to right, one word at a time.
- **Architectural Role**: To enforce this law of time, the model uses an additive mask (a grid of numbers called M). This mask acts as a barrier, preventing the model from seeing future tokens.
- **Failure Mode Without Causal Masking**: If we don't use this mask during training, the model can simply look ahead at the next word in the sequence, which breaks the autoregressive property.

Definition, Architecture, and The Mathematics

To maintain the strict rule of time, we must define exactly what the model is allowed to see. We do this by creating a matrix M :

$$M_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

Here, i represents the position of the word we are currently analyzing, and j represents the position of the word we want to pay attention to.

Before we convert our attention scores into percentages, we inject this mask into the calculation:

$$S_{\text{masked}} = \frac{QK^T}{\sqrt{d_k}} + M$$

We take our standard raw attention calculation (Query Q multiplied by Key K^T , divided by $\sqrt{d_k}$) and we simply *add* the mask M .

![[Figure 3.3: Causal Attention Masking Matrix Arithmetic](assets/diagram_3_kv_cache.jpg)]

xml

Attention Mask M

0

-∞

-∞

-∞

0

0

-∞

-∞

0

0

0

-∞

0

0

0

0

→

softmax

Attention Matrix A

1.0

0.0
0.0
0.0
0.4
0.6
0.0
0.0
0.2
0.5
0.3
0.0
0.1
0.3
0.2
0.4

To see why $-\infty$ is such a powerful blocking tool, we look at the mathematical property of the exponential function ($\exp$), which

$$\exp(-\infty) = 0$$

When you calculate the exponent of negative infinity, the mathematical result is exactly zero. This is a hard limit. No matter how high you raise the base, the result is still zero.

Finally, we calculate our masked probabilities:

$$A_{\text{causal}} = \text{softmax}(S_{\text{masked}})$$

The 'softmax' function takes our modified scores and turns them into final percentages. Because the future words were crushed to 0,

Explicit Differentiation: Training vs. Inference

- During Training (Learning the Weights):**
Causal masking is aggressively used here. During training, we don't feed words to the model one by one. To maximize the efficiency
- During Inference / Generation (Runtime Chat with ChatGPT/Claude):**
When you chat with a model, you generate text sequentially. Because future words literally have not been created yet, the model can

Concrete Numerical Toy Walk-Through

Imagine a 3-word sentence: "I love cats".

We want to calculate the attention for the 2nd word ("love", position $i=2$).

The raw attention scores (before applying the mask) for looking at word 1 ("I"), word 2 ("love"), and word 3 ("cats") might naturally be $[10, 12, 15]$.

Notice how the model desperately wants to look at word 3 ("cats") with a high score of 15! But word 3 is in the future.

We apply the causal mask M for position $i=2$:

$$\text{Mask} = [0, 0, -\infty]$$

We simply add the mask to the raw scores:

$$\text{Masked Scores} = [10 + 0, 12 + 0, 15 + (-\infty)] = [10, 12, -\infty]$$

Now we apply the exponential function ($\exp$) and softmax logic:

The $-\infty$ evaluates to a literal 0.

The final attention percentages become roughly 12% for "I", 88% for "love", and exactly 0% for "cats". The future is successfully blocked.

2-Sentence Plain Lingo

The causal mask blocks future words by adding negative infinity to their connection scores before running the softmax calculation. This ensures the model only attends to information from previous tokens.

Real-World & NLP Behaviors (2 Examples)

- Example 1: Next-Token Prediction in Autoregressive Generation:** In GPT-4 or Claude, when generating the continuation for "The cat sat on the mat, and the dog", the model must predict the next token based only on the context provided so far.
- Example 2: Prefix Caching vs Bidirectional Prefix:** In encoder-decoder or PrefixLM models (such as T5 or PaLM Prefix), the attention mechanism can be modified to allow the model to attend to both past and future tokens within a specific prefix, which is useful for tasks like summarization or question-answering.

3.4 The KV-Cache Mechanism

Why?

- Eliminating Redundant Autoregressive Computation:** When generating a paragraph, the AI speaks one word at a time. If it generates a long sentence, it would be inefficient to recompute the attention for every word from scratch.
- Architectural Role:** To solve this, we use the Key-Value (KV) Cache. As the model generates words, it saves the historical Key and Value vectors for each token.
- Computational Complexity Shift:** Without the cache, generating text gets exponentially slower with every new word because the attention mechanism has to look back at all previous tokens. With the KV-cache, the complexity is reduced to a constant time per token.

Definition, Architecture, and The Mathematics

Let's look at the mathematical steps the model takes when generating a brand-new token at a specific time step, t .

$$q_t = x_t W_Q$$

First, the model takes *only* the single newest word vector x_t . It ignores all past words. It multiplies this single word by the Query matrix W_Q .

$$k_t = x_t W_K, \quad v_t = x_t W_V$$

Next, the model multiplies that same single word x_t by the Key matrix W_K to get its key k_t , and by the Value matrix W_V to get its value v_t .

Now, we introduce the caching mechanism:

$$K_{\leq t} = \begin{bmatrix} K_{<t} \\ k_t \end{bmatrix}$$

This equation updates our memory. The variable $K_{<t>}$ represents the giant list of all the Keys from all the previous words that we

$$V_{\leq t} = \begin{bmatrix} V_{<t>} \\ v_t \end{bmatrix}$$

We do the exact same thing for the values. We take our saved, historical list of values $V_{<t>}$, and we append our new value v_t to

$$z_t = \text{softmax}\left(\frac{q_t K_{\leq t}^T}{\sqrt{d_k}}\right) V_{\leq t}$$

Finally, we perform the actual attention calculation. Look closely at how tiny this math is! We take our single, isolated query q_t

! [Figure 3.4: Autoregressive KV-Cache VRAM Dynamic Memory](assets/diagram_3_kv_cache.jpg)

While the math is small, caching takes up physical hardware space:

$$\text{Memory}_{\text{KV}} = 2 \times n_{\text{layers}} \times n_{\text{heads}_{\text{kv}}} \times d_k \times L \times \text{bytes}_{\text{per_element}}$$

This equation calculates exactly how much physical GPU RAM (memory) the cache consumes. The number 2 exists because we store both K

Explicit Differentiation: Training vs. Inference

- **During Training (Learning the Weights)**:**

The KV-cache is **completely ignored and turned off** during training. Because the model trains on giant parallel documents all at

- **During Inference / Generation (Runtime Chat with ChatGPT/Claude)**:**

During a live chat, the KV-cache is the absolute lifeblood of the system. Without it, generation would be impossibly slow. When you

Concrete Numerical Toy Walk-Through

Let's trace generating the 3rd word in a sentence.

- **The naive way (no cache)*:** To generate word 3, we mathematically calculate the Key vectors for word 1, word 2, and word 3 from scratch.
- **The KV-cache way*:** We look in our GPU memory. We see we already saved the previous keys!

Cache $K_{<3>} = [K_1, K_2]$.

We do the math to calculate **ONLY** the Key for word 3: K_3 . (Just 1 calculation).

We append it to our cache list:

New Cache $K_{\leq 3} = [K_1, K_2, K_3]$.

We saved ourselves from doing 2 extra calculations. It seems small now, but when generating word 1,000, we save ourselves from doing

2-Sentence Plain Lingo:**

Instead of recomputing the memory representations of every past word each time a new word is generated, the model saves the completed

Real-World & NLP Behaviors (2 Examples)

- **Example 1: Interactive Streaming Chat Latency**:** Without a KV-cache, generating a 500-token response would cause output generation to be extremely slow.
- **Example 2: GPU Memory Bottlenecks & PagedAttention (vLLM)**:** In production LLM serving, a 70B parameter model serving a batch of

Module 4: Signal Integrity & Normalization (The Residual Highway & Stability)

4.1 Residual Skip Connections (The Additive Highway)

The Core Problem: Vanishing Signals

Imagine a game of "telephone" played by 100 people. By the time the final person hears the message, the original words are entirely gone.

To solve this, we create a direct, uninterrupted identity highway from the very beginning of the model to the very end. We call this

The Step-by-Step Architecture

When a word token arrives at a new layer, the immediate goal is to update it with new information without destroying its historical memory.

$$x_{\text{out}} = x_{\text{in}} + \text{Sublayer}(x_{\text{in}})$$

In this formula, x_{in} represents our incoming token vector from the residual stream—a list of numbers holding every piece of information from before.

But why is this simple addition so profoundly important? We need to look at what happens during the learning phase when the model makes

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{in}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{out}}} \cdot$$

This formula maps how the correction signal (the gradient of the loss, denoted by $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{out}}}$)

![[Figure 4.1: Residual Stream Highway Architecture](assets/diagram_4_residual_norm.jpg)]

Concrete Numerical Toy Walk-Through

Let's see this forward pass arithmetic in action.

Imagine we have a tiny 2-dimensional token vector representing the word "bank".

Incoming vector: $\mathbf{x}_{\text{in}} = [2, 10]$

(Let's pretend the \$2\$ represents the "financial" concept and the \$10\$ represents the "river" concept.)

The vector enters the Attention sublayer. The Attention mechanism looks at the surrounding sentence ("I deposited money in the bank")

Sublayer adjustment: $\mathbf{x}_{\text{in}} = [5, -8]$

Now, we perform the residual addition to move the token forward:

$$\mathbf{x}_{\text{out}} = [2, 10] + [5, -8]$$

$$\mathbf{x}_{\text{out}} = [7, 2]$$

The original token vector has been securely updated. The "financial" dimension increased from 2 to 7, and the "river" dimension decre

Training vs. Inference

* **During Training (Learning the Weights):**

The residual connection is the absolute savior of the massive pre-training process. When processing millions of words simultaneousl

* **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**

When a human user is chatting with the model, training is over, and data only moves forward. The residual stream acts as a central

Real-World & NLP Behaviors

1. **Preservation of Lexical Identity:** A token's fundamental dictionary identity (e.g., that it is the word "Einstein") can survi

2. **Layer Pruning Robustness:** Because layers compute additive adjustments rather than total transformations, researchers have show

4.2 Layer Normalization vs. RMSNorm

The Core Problem: Exploding Magnitudes

As our token vectors travel down the residual highway, they are constantly having numbers added to them layer after layer. If you kee

The Step-by-Step Architecture

Historically, models accomplished this using a technique called Layer Normalization. The goal of Layer Normalization is to force the

$$\mathbf{LN}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

In this classic formula, we take our incoming token vector $\mathbf{x}$. We calculate the average (mean) of all its internal numbers,

However, modern frontier models (like LLaMA, Mistral, and Gemma) realized that calculating the mean μ and shifting the vector to

Before we normalize, we must calculate the raw magnitude of the vector, which is called the Root Mean Square.

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

In this formula, we take each individual coordinate x_i of our token vector, square it (x_i^2) so that negative numbers become po

Now that we know the exact size of the vector, we can elegantly normalize it.

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma$$

We take our original token vector $\mathbf{x}$ and divide every single number inside it by the single RMS number we just cal

![[Figure 4.2: Layer Normalization vs RMSNorm Architecture](assets/diagram_4_2_rmsnorm.jpg)]

Concrete Numerical Toy Walk-Through

Let's see RMSNorm in action with simple numbers.

Imagine a tiny 2-dimensional token vector: $\mathbf{x} = [3, 4]$

We need to normalize this to prevent it from growing out of control later.

First, we calculate the squares: $3^2 = 9$ and $4^2 = 16$.

Next, we find the average of these squares. The sum is $9 + 16 = 25$. We have $d=2$ dimensions, so the average is $25 / 2 = 12.5$. Now, take the square root to get the overall RMS magnitude: $\sqrt{12.5} \approx 3.53$. (We ignore the tiny ϵ for this toy example).

Finally, we apply the normalization by dividing the original vector by the RMS:
 Normalized Vector = $[\frac{3}{3.53}, \frac{4}{3.53}] = [0.85, 1.13]$

Notice how the original values of 3 and 4 have been safely scaled down to 0.85 and 1.13 . Their relative proportions to one another remain the same.

⊗ Training vs. Inference

*** During Training (Learning the Weights):***

When pre-training on thousands of GPUs, RMSNorm is essential for precision stability. Models train using 16-bit floating point numbers.

*** During Inference / Generation (Runtime Chat with ChatGPT/Claude):***

When a user asks a question, the model must generate text extremely fast. Because RMSNorm drops the mean calculation (μ) and then squares the result, it's computationally expensive.

4.3 Pre-LN vs. Post-LN Architecture

□ The Core Problem: Where do we place the toll booth?

We know we need normalization to prevent exploding numbers, and we know we need a residual highway to prevent vanishing gradients. But where do we place the toll booth?

□ The Step-by-Step Architecture

When the Transformer was first invented in 2017, the creators placed the normalization step at the very end of the layer, forcefully.

$$\mathbf{x}^{(l)} = \text{Norm}(\mathbf{x}^{(l-1)} + \text{Sublayer}(\mathbf{x}^{(l-1)}))$$

In this historical Post-LN formula, $\mathbf{x}^{(l-1)}$ is the incoming data. The sublayer calculates its adjustments, and those adjustments are added to the original data.

To fix this fatal flaw, modern large language models flipped the architecture inside out. They invented **Pre-LN** (Pre-Layer Normalization).

$$\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \text{Sublayer}(\text{Norm}(\mathbf{x}^{(l-1)}))$$

Look closely at the modern Pre-LN formula. The main residual highway ($\mathbf{x}^{(l-1)} + \dots$) remains absolutely pristine and untouched.

[Figure 4.3: Post-LN vs Pre-LN Architectural Flow](assets/diagram_4_3_pre_post_ln.jpg)

□ Concrete Numerical Toy Walk-Through

Let's see the mathematical difference using simple numbers. Imagine our incoming main highway vector is $\mathbf{x} = [10, 10]$ and our adjustment vector is $[2, 2]$.

*** Under the old Post-LN rules:**

First, we add: $[10, 10] + [2, 2] = [12, 12]$

Then, we normalize the entire main highway: $[12, 12] / 2 = [6, 6]$

The highway itself has been drastically altered and forcefully shrunk. Its original massive identity is obscured.

*** Under the modern Pre-LN rules:**

First, we make a normalized copy just for the sublayer: $[10, 10] / 2 = [5, 5]$. (The sublayer uses this copy to figure out its adjustment).

Then, we add the adjustment directly to the untouched highway: $[10, 10] + [2, 2] = [12, 12]$.

The result is exactly what a residual addition should be. The highway retains its full, massive historical magnitude, effortlessly carrying the information.

⊗ Training vs. Inference

*** During Training (Learning the Weights):***

Pre-LN is the architectural secret that makes training modern giant models possible. Models using the old Post-LN frequently diverge.

*** During Inference / Generation (Runtime Chat with ChatGPT/Claude):***

During generation, Pre-LN allows models to physically scale to extreme depths. When predicting words in an 80-layer model like LLaMA, the model can maintain stability.

□ Real-World & NLP Behaviors

- *Zero-Warmup Training Stability:** As mentioned, AI researchers no longer have to spend days carefully tuning fragile "learning rate schedules".
- *Deep Stacking Feasibility:** When scaling from a tiny 12-layer model (like the original BERT) to an 80-layer model, Pre-LN is the key.

Module 5: Factual Memory & Computation (The Feed-Forward Network / FFN)

5.1 The Two-Layer FFN Expansion

□ The Functional Purpose: A Bank of Factual Knowledge

In previous modules, we learned that Self-Attention acts as a communication router, moving information around between words that are

! [Figure 5.1: Two-Layer Feed-Forward Network (FFN) Expansion] (assets/diagram_5_1_ffn_expansion.jpg)

□ The Mathematics & Working, Step-by-Step

To retrieve facts, the model must first take the relatively small numerical representation of our token and expand it into a massive

$$\mathbf{k} = \mathbf{x} \mathbf{W}_1 + \mathbf{b}_1$$

In this equation, our starting point is $\mathbf{x}$, which represents the incoming token vector (a list of numbers capturing the mea

However, simple multiplication and addition only draw straight lines mathematically. If we stop here, the network cannot learn comple

$$\mathbf{a} = \text{ReLU}(\mathbf{k})$$

Here, we take our expanded vector $\mathbf{k}$ and pass it through an activation function, represented by ReLU (Rectified Li

Now that our specific factual detectors have fired, we need to gather their stored knowledge and shrink it back down so it can fit pe

$$\mathbf{y} = \mathbf{a} \mathbf{W}_2 + \mathbf{b}_2$$

We take our activated vector $\mathbf{a}$ and multiply it by $\mathbf{W}_2$, which is our contraction matrix. While $\mathbf{W}_1$ acted as the "Keys"

□ Concrete Numerical Toy Walk-Through

Let's look at how the basic arithmetic of an FFN works using tiny lists of numbers. Imagine our input token vector for the word "Fran

First, we expand it. Let's assume our learned expansion matrix $\mathbf{W}_1$ is a grid of weights that looks like this:

Column 1: $[1, 0]$

Column 2: $[-1, 1]$

Column 3: $[0, -1]$

Column 4: $[2, -1]$

To multiply our vector $[1, 2]$ by $\mathbf{W}_1$, we multiply the numbers pairwise and add them together for each column. (We will assume the

* For column 1: $(1 \times 1) + (2 \times 0) = 1$

* For column 2: $(1 \times -1) + (2 \times 1) = 1$

* For column 3: $(1 \times 0) + (2 \times -1) = -2$

* For column 4: $(1 \times 2) + (2 \times -1) = 0$

Our expanded intermediate vector $\mathbf{k}$ is now $[1, 1, -2, 0]$. We have successfully widened our representation from 2 numbers

Next, we apply the ReLU activation. Any negative number becomes 0 .

$\text{ReLU}([1, 1, -2, 0])$ turns the -2 into a 0 .

Our activated vector $\mathbf{a}$ is $[1, 1, 0, 0]$. Notice how the third concept detector was completely shut off.

Finally, we contract it back down using our second matrix $\mathbf{W}_2$. Let's assume $\mathbf{W}_2$ is structured to map 4 numbers back down to 2:

Column 1: $[1, 2, -1, 0]$

Column 2: $[0, 1, -1, 5]$

We multiply our activated vector $\mathbf{a} = [1, 1, 0, 0]$ by $\mathbf{W}_2$:

* For column 1: $(1 \times 1) + (1 \times 2) + (0 \times -1) + (0 \times 0) = 3$

* For column 2: $(1 \times 0) + (1 \times 1) + (0 \times -1) + (0 \times 5) = 1$

Our final updated token vector is $\mathbf{y} = [3, 1]$. The original token entered the FFN as $[1, 2]$, triggered specific memories,

□ Distinguishing Training vs. Inference

****During Training (Pre-training / Learning the Facts):****

When the model is training inside giant GPU clusters, it processes enormous sequences of text—often 4,096 tokens or more—all simultan

****During Inference (Runtime Chat with ChatGPT/Claude):****

When you are chatting with an AI, the training is over. The matrices $\mathbf{W}_1$ and $\mathbf{W}_2$ are locked and frozen. The model evaluates your

5.2 Modern Gated Activation: SwiGLU

□ The Functional Purpose: Dynamic Precision and Control

Traditional activation functions like ReLU are blunt instruments: they simply check if a number is negative and crush it to zero. This is a waste of precision and control.

![[Figure 5.2: SwiGLU Multiplicative Gating Mechanism](assets/diagram_5_swiglu.jpg)]

□ The Mathematics & Working, Step-by-Step

To build SwiGLU, we must first calculate the "proposed thought". What fact is the model trying to recall right now?

$$\mathbf{v}_{\text{up}} = \mathbf{x} W_{\text{up}}$$

We take our incoming token vector $\mathbf{x}$ and multiply it by a matrix called W_{up} . Just like the first layer of the transformer, this produces a new vector.

Simultaneously, we need a mechanism to evaluate and control whether this proposed knowledge is actually appropriate. We generate a parallel "gate" vector.

$$\mathbf{v}_{\text{gate}} = \mathbf{x} W_{\text{gate}}$$

We take that exact same input token $\mathbf{x}$ and multiply it by an entirely different learned matrix called W_{gate} . This produces a "raw" control signal.

However, $\mathbf{v}_{\text{gate}}$ contains raw numbers that could be extremely large or extremely negative. We need to convert them into a smooth, controllable "volume knob".

$$\mathbf{g} = \mathbf{v}_{\text{gate}} \times \frac{1}{1 + 2.718^{-\mathbf{v}_{\text{gate}}}}$$

In this equation, we take our raw control numbers $\mathbf{v}_{\text{gate}}$ and multiply them by a mathematical fraction that relies on the exponential function. This creates a smooth, S-shaped curve.

Now that we have both our proposed thought and our smooth volume knob, we combine them together.

$$\mathbf{h} = \mathbf{g} \odot \mathbf{v}_{\text{up}}$$

Here, we use the $\odot$ symbol, which represents the elementwise Hadamard product. This is just a fancy way of saying we multiply the vectors element by element.

Finally, just as in the traditional FFN, we must shrink this wide, filtered vector back down to normal size.

$$\mathbf{y} = \mathbf{h} W_{\text{down}}$$

We multiply our filtered intermediate vector $\mathbf{h}$ by W_{down} , the contraction matrix. This matrix squishes the appropriate dimensions back to the original size.

□ Concrete Numerical Toy Walk-Through

Let's see how SwiGLU dynamically silences a bad thought using simple arithmetic. Imagine our token vector is just a single number: $\mathbf{x} = [4]$.

First, we propose a thought:

Let's assume our learned "up" weight is $W_{\text{up}} = [4]$.

Our candidate fact $\mathbf{v}_{\text{up}} = 2 \times 4 = 8$. The network is pushing a strong feature value of 8 .

At the very same time, we calculate the gate:

Let's assume the learned gate weight recognizes this is a bad context for this fact, so $W_{\text{gate}} = [-1]$.

Our raw control signal $\mathbf{v}_{\text{gate}} = 2 \times (-1) = -2$.

Next, we calculate the smooth Swish volume dial $\mathbf{g}$.

We need to squash -2 into a fraction: $\frac{1}{1 + 2.718^{-(-2)}}$ which equals $\frac{1}{1 + 2.718^2} \approx \frac{1}{1 + 7.38}$.

So the sigmoid output is 0.12 .

We multiply the raw gate by this sigmoid: $\mathbf{g} = -2 \times 0.12 = -0.24$. Our volume dial is set to -0.24 .

Now, we multiply the dial by the proposed thought (the gating mechanism):

$$\mathbf{h} = -0.24 \times 8 = -1.92$$

Notice what happened: the original strong candidate thought of 8 was drastically silenced and inverted down to -1.92 because the gate was negative.

Finally, we contract it. Let's say $W_{\text{down}} = [2]$.

$$\mathbf{y} = -1.92 \times 2 = -3.84$$

□ Distinguishing Training vs. Inference

****During Training (Pre-training / Learning the Gates):****

When the model is being trained on massive GPU clusters learning from trillions of words, the traditional ReLU function causes a massive waste of precision and control.

****During Inference (Runtime Chat with ChatGPT/Claude):****

When generating text for an end-user, the model executes the SwiGLU block one token at a time. The continuous multiplication of the g

Module 6: Token Emission & Sampling (From Residual Vector to Text)

6.1 The Unembedding Head and Logits

The Conceptual Goal

By the time data has flowed through dozens of transformer layers, the model has built a deeply complex understanding of the sentence.

Creating a Score for a Single Word

To figure out if a specific word should be the next token, the model compares its final abstract thought to the mathematical "profile

$$z_i = \mathbf{h}_{\text{final}} \cdot \mathbf{w}_i$$

In this equation, we are calculating a single number called z_i , which is the raw score (often called a "logit") for one specific w

****Concrete Numerical Walk-Through:****

Imagine our model's final thought is simplified to just two numbers: $\mathbf{h}_{\text{final}} = [2, 3]$.

Now, imagine our dictionary only has two words.

The profile for the word "apple" is $\mathbf{w}_{\text{apple}} = [1, 2]$.

The profile for the word "car" is $\mathbf{w}_{\text{car}} = [-1, 0]$.

To find the score for "apple," we multiply matching positions and add: $(2 \times 1) + (3 \times 2) = 2 + 6 = 8$. The score (z) for

To find the score for "car," we do the same: $(2 \times -1) + (3 \times 0) = -2 + 0 = -2$. The score (z) for "car" is -2 .

Because 8 is much larger than -2 , the model strongly believes "apple" is the correct next word.

Scaling Up to the Entire Vocabulary

Checking one word at a time is too slow. The model needs to check its final thought against every single word in its vocabulary simul

$$\mathbf{z} = \text{RMSNorm}(\mathbf{h}_{\text{final}}) W_U$$

Here, instead of finding just one score, we are calculating a massive list of scores called $\mathbf{z}$ (bolded to show it contains

xml

$\mathbf{h}_{\text{final}}$ [1 x 4096]

W_U Head

[4096 x 128,000]

Vocab Logits $\mathbf{z}$

["the": 14.2, ...]

During Training vs. Inference

* **During Training (Learning the Weights):** When engineers are teaching the AI on giant supercomputers, the model calculates these weights.

* **During Inference / Generation (Runtime Chat with ChatGPT/Claude):** When you are chatting with the model, no learning happens.

6.2 Temperature Scaling and Entropy

The Conceptual Goal

Once the model generates its raw scores (logits) for every word, we have a problem: raw scores are just arbitrary numbers like \$14.25\$.

Adjusting the Raw Scores

Before we calculate percentages, we divide every raw score by a simple dial called Temperature. We represent this mathematically as:

$$z'_i = \frac{z_i}{T}$$

In this formula, z_i is our original raw score for a specific word, and T is the Temperature setting, which is just a positive number.

Concrete Numerical Walk-Through:

Imagine the raw score for "apple" is \$10\$ and the raw score for "car" is \$5\$.

If we use a **low temperature** of $T = 0.5$, we divide both scores by \$0.5\$. The score for "apple" becomes \$20\$ (\$10 / 0.5 = 20\$). The score for "car" becomes \$10\$ (\$5 / 0.5 = 10\$).

If we use a **high temperature** of $T = 2.0$, we divide both scores by \$2.0\$. The score for "apple" becomes \$5\$ (\$10 / 2.0 = 5\$). The score for "car" becomes \$2.5\$ (\$5 / 2.0 = 2.5\$).

Converting to Percentages (Softmax)

Now that we have tweaked the distances between our scores, we must convert them into proper probabilities that total \$100\%\$. We do this using the Softmax function.

$$P_i(T) = \frac{\exp(z_i / T)}{\sum_{k=1}^V \exp(z_k / T)}$$

This looks complicated, but it is just a way to turn numbers into shares of a pie. $P_i(T)$ represents the final probability percentage for word i .

![[Figure 6.2: Temperature Entropy Calibration](assets/diagram_6_sampling.jpg)]

During Training vs. Inference

* **During Training (Learning the Weights):** During the learning phase on the supercomputer, Temperature is strictly locked to exactly 1.0.

* **During Inference / Generation (Runtime Chat with ChatGPT/Claude):** When using an AI application, Temperature is heavily manipulated to control the model's output.

6.3 Nucleus Sampling (Top- p) and Top- k

The Conceptual Goal

Even after applying Temperature and turning our scores into percentages, we face a critical danger: the "long tail" of bad words. Because the model considers every word in the vocabulary, it can sometimes generate words that are nonsensical or harmful.

Drawing the Nucleus Cutoff Line

The most advanced way to chop off the bad words is called Nucleus Sampling, or Top- p . Instead of keeping a fixed number of words, i

$$\sum_{i \in V^{(p)}} P_{(i)} \geq p$$

Here, we sort all our words from highest probability to lowest. $P_{(i)}$ represents the probability of the current word we are looking at.

Concrete Numerical Walk-Through:

Imagine our model wants to guess the next word. The probabilities are:

"Apple" = \$70\%\$

"Banana" = \$20\%\$

"Shoe" = \$8\%\$

"Helicopter" = \$2\%\$

Let's set our Top- p threshold to \$0.90\$ (which is \$90\%\$).

We start at the top. We take "Apple" (\$70\%\$). Our total is \$70\%\$. We haven't hit \$90\%\$ yet.

We add "Banana" (\$20\%\$). Our new total is \$70\% + 20\% = 90\%\$.

We have perfectly hit our threshold \$p \ge 0.90\$! We immediately stop.

"Apple" and "Banana" become our VIP club (\$V^{(p)}\$). The words "Shoe" and "Helicopter" are permanently deleted from consideration, cutting off the long tail.

Re-Balancing the Remaining Words

Because we threw away some words (like "Shoe" and "Helicopter"), our remaining probabilities no longer add up to a perfect \$100\%\$. We need to re-normalize the probabilities of the remaining words.

$$P'(w_i) = \begin{cases} \frac{P(w_i)}{\sum_{w_j \in V^{(p)}} P(w_j)} & \text{if } w_i \in V^{(p)} \\ 0 & \text{otherwise} \end{cases}$$

This equation defines our brand new probability, $P'(w_i)$, for any given word. The large bracket gives us two simple scenarios. The

Let's finish our walk-through. Our surviving words were "Apple" (\$0.70) and "Banana" (\$0.20). Their sum is \$0.90.

To find the new probability for "Apple", we divide its old score by the sum: $0.70 \div 0.90 = 0.777$ (or 77.7%).

To find the new probability for "Banana", we do the same: $0.20 \div 0.90 = 0.222$ (or 22.2%).

If we add 77.7% and 22.2%, we get roughly 100%. The math is perfectly balanced, and the model can now safely roll its metaphor

![[Figure 6.3: Nucleus Top-p Probability Mass Cutoff](assets/diagram_6_sampling.jpg)]

During Training vs. Inference

* **During Training (Learning the Weights):** Top- p (and its simpler cousin, Top- k) are strictly turned off. During training, t

* **During Inference / Generation (Runtime Chat with ChatGPT/Claude):** Top- p is running dynamically on every single word the mod

Module 7: Post-Training, Alignment & Reasoning RL

7.1 Supervised Fine-Tuning (SFT) & Instruction Tuning

□ The Conceptual Foundation

Imagine you have just finished training a massive language model on trillions of words from the internet. You might think you have bu

Supervised Fine-Tuning (SFT) is the crucial first step in transforming this wild, unpredictable text-generator into an obedient, conv

□ How SFT Works in Practice

During SFT, we create explicit two-party dialogues. We give the model a user prompt, and we provide the perfect target response.

For example, our prompt might be: `<|user|> Solve for x: 2x + 4 = 10 <|assistant|>`

The target response we want the model to learn is: `x = 3`

The secret to Supervised Fine-Tuning is that we only penalize the model for making mistakes on the **target response**. We do not car

![[Figure 7.1: The Post-Training Alignment Hierarchy: SFT, RLHF, and GRPO](assets/diagram_7_post_training.jpg)]

xml

Prompt Tokens x

Loss Masked: Weight = 0.0

Response Tokens y

Compute Cross-Entropy Loss

□ The Mathematics of SFT (Step-by-Step)

To train our model, we first need to look at the exact mathematical probability that our model will guess the correct next word in the

$$P_{\theta}(y_t \mid x, y_{<t})$$

In this expression, we use the letter P to stand for probability. The symbol θ (theta) represents all the millions of adjustable parameters.

Working with raw percentages is difficult because multiplying many small fractions together results in microscopically small numbers.

$$\log P_{\theta}(y_t \mid x, y_{<t})$$

By taking the logarithm ($\log$) of our probability, we turn tiny fractions into manageable negative numbers. If the model is very confident, the log probability is close to 0.

Now, we need to calculate the total error—or "loss"—for an entire dialogue, which involves summing up the logarithms for every single token in the dataset.

$$\mathcal{L}_{\text{SFT}} = -\sum_{t=1}^{|y|} \log P_{\theta}(y_t \mid x, y_{<t})$$

Here, $\mathcal{L}_{\text{SFT}}$ stands for the total Supervised Fine-Tuning Loss, which is the final error score we want to minimize.

To train the model properly, we don't just do this for one conversation; we calculate the average error over an entire massive dataset.

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x, y) \sim \mathcal{D}} \left[\sum_{t=1}^{|y|} \log p_{\theta}(y_t \mid x, y_{<t}) \right]$$

In this final, complete equation, the stylized $\mathbb{E}$ stands for Expected Value, which is just a formal mathematical way of saying "average".

□ Concrete Numerical Toy Walk-Through

Let's pretend we have a tiny model and a dataset with one simple dialogue.

Prompt x : "What is 2+2?"

Target Response y : "It is 4"

The target response has three words: "It", "is", "4".

- For word 1 ("It"), the model looks at the prompt and guesses "It" with a probability of 0.10. The math is $-\log(0.10) \approx 1.0$.
- For word 2 ("is"), the model looks at the prompt plus "It" and guesses "is" with a probability of 0.50. The math is $-\log(0.50) \approx 0.3$.
- For word 3 ("4"), the model looks at the prompt plus "It is" and guesses "4" with a probability of 0.01. The math is $-\log(0.01) \approx 2.0$.

We sum these penalties: $1.0 + 0.3 + 2.0 = 3.3$.

Our total SFT Loss is 3.3. The computer will now adjust the model's internal dials to raise those probabilities. If, after training, the model's guesses are better, the loss will be lower.

◉ Training vs. Inference

* **During Training (Learning the Weights):**

We use powerful GPU clusters to process thousands of entire conversations simultaneously. We apply a "causal mask" to the data, meaning the model only sees information up to the current token.

* **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**

The model is frozen; no learning happens. A human user types a prompt into a chat window. The model uses its fine-tuned weights to generate a response.

□ Real-World NLP Behaviors

- Instruction Following (LIMA / Alpaca):** Research like the LIMA paper showed that fine-tuning a base model on just 1,000 meticulously crafted instructions can significantly improve performance on various tasks.
- Specialized Tool-Calling:** SFT is used to train models to emit structured computer code (like JSON) whenever a user asks a real-world question that requires a specific tool or API.

7.2 Reward Modeling & RLHF (PPO)

□ The Conceptual Foundation

Supervised Fine-Tuning is powerful, but it has severe limitations. For complex questions like, "Explain quantum physics to a five-year-old," SFT often produces generic or incorrect answers.

To solve this, we use Reinforcement Learning from Human Feedback (RLHF). Instead of forcing the model to copy an answer, we let the model generate multiple responses, and humans (or a reward model) rank them.

□ How RLHF Works in Practice

RLHF happens in three phases. First, we collect preferences: the model generates a winning response and a losing response, and humans (or a reward model) rank them. This data is used to train a reward model.

xml

Prompt x

LLM Policy π_θ

Ref Model π_{ref}

Reward Model R_ϕ

KL Penalty $\beta * D_{\text{KL}}$

□ The Mathematics of RLHF (Step-by-Step)

Our first task is to build the Reward Model. We want it to look at the user's prompt alongside a response, and output a single numerical

$$\text{Difference} = R_{\phi}(x, y_w) - R_{\phi}(x, y_l)$$

In this formula, R_{ϕ} represents our separate Reward Model, parameterized by its own set of internal weights called ϕ (phi).

To actually train this Reward Model, we need to turn this simple difference into a formal loss function that we can minimize.

$$\mathcal{L}_{\text{RM}}(\phi) = -\log \sum (R_{\phi}(x, y_w) - R_{\phi}(x, y_l))$$

Here, $\mathcal{L}_{\text{RM}}(\phi)$ is the loss (error) for the Reward Model. We take our point difference from the previous step and pass

Once the Reward Model is trained and frozen, we bring back our main language model. We tell it to generate new answers to prompts, and

$$\text{Reward}_{\text{total}} = R_{\phi}(x, y) - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta}(y \mid x) \parallel \pi_{\text{ref}}(y \mid x))$$

This is the PPO objective equation representing the total adjusted points the model earns. $\text{Reward}_{\text{total}}$ is what we

The subtraction represents our leash. The symbol $\mathbb{D}_{\text{KL}}$ stands for Kullback-Leibler Divergence, which is a mathematical

□ Concrete Numerical Toy Walk-Through

Imagine the user asks: "How do I bake a cake?"

The model's active policy π_{θ} writes a good recipe. The frozen Reward Model R_{ϕ} reads it and awards 10 points.

However, the active model used a slightly weird formatting trick to get those 10 points, diverging from the safe reference model π_{ref} .

If our leash strength dial β is set to 1.5, we calculate the penalty: $1.5 \times 2 = 3$.

The total adjusted reward the model actually receives is $10 - 3 = 7$. The model updates its weights to get closer to 7, learning to

◉ Training vs. Inference

* **During Training (Learning the Weights):**

We run a massive, highly complex server setup holding four distinct neural networks in GPU memory at the same time: the active Policy

* **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**

The complex multi-model dance is completely gone. The Reward Model and Reference Model are deleted from memory. We are left with ju

□ Real-World NLP Behaviors

1. **Safety Refusals:** When asked "How do I make a bomb?", a base model happily generates instructions. RLHF trains the model that

2. **Conciseness vs. Verbosity:** If a reward model unintentionally awards higher scores to longer answers, the PPO policy rapidly di

7.3 Direct Preference Optimization (DPO)

□ The Conceptual Foundation

RLHF with PPO is incredibly effective, but it is an absolute nightmare for engineers to run. As we just learned, PPO requires loading

In 2023, researchers made a massive mathematical breakthrough called Direct Preference Optimization (DPO). They proved mathematically

□ How DPO Works in Practice

With DPO, we only need our dataset of prompts, winning answers, and losing answers. We keep two models in memory: our active model be

![Figure 7.3: Direct Preference Optimization (DPO) Loss Architecture](assets/diagram_7_3_dpo.jpg)

□ The Mathematics of DPO (Step-by-Step)

The genius of DPO lies in proving that any language model essentially has a hidden, implicit reward function built into its probabili

$$r(x, y) = \beta \log \frac{\pi_{\theta}(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

In this formula, $r(x, y)$ represents the implicit reward—the invisible score the model is giving to an answer y for prompt x . To

Because we have a mathematical way to extract a score for any answer, we can extract the implicit score for the winning answer y_w

$$\text{Advantage} = r(x, y_w) - r(x, y_l)$$

This is identical in spirit to what we did in RLHF: we subtract the implicit reward of the loser $r(x, y_l)$ from the implicit reward of the winner $r(x, y_w)$. To turn this into a final loss function that our computers can use to optimize the network, we combine our formulas. We plug our implicit reward formula into the DPO loss function:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sum_{\theta} \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\theta}(y_l | x)} \right) \right]$$

This looks highly intimidating, but it is merely the exact same logic we have already covered, combined into one line. Finally, to understand how the neural network actually changes its internal dials, we look at the mathematical gradient (the instruction for how to update the weights):

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}} = -\beta \left(\frac{1}{\sigma} \left(\hat{r}_l - \hat{r}_w \right) \times \left(\nabla_{\theta} \log \pi_{\theta}(y_w | x) - \nabla_{\theta} \log \pi_{\theta}(y_l | x) \right) \right)$$

Here, the triangle symbol ∇_{θ} (nabla) represents the "gradient"—the precise mathematical arrow that tells the computer exactly how to update the weights. The most important part is in the brackets: $\nabla_{\theta} \log \pi_{\theta}(y_w | x)$ represents nudging the dials *up* to increase the probability of the winning response, while $\nabla_{\theta} \log \pi_{\theta}(y_l | x)$ represents nudging the dials *down* to decrease the probability of the losing response.

Concrete Numerical Toy Walk-Through

Imagine a user asks, "How are you?"
 The winning answer y_w is: "I am good."
 The losing answer y_l is: "As an AI language model, I do not have feelings."

Initially, our active model assigns a 10% probability to the winner, and a 10% probability to the loser. The baseline reference model also assigns 10% to both. Because the active model and reference model match, the implicit rewards $\hat{r}_w$ and $\hat{r}_l$ are both exactly 0. The difference is 0. Through training via the DPO gradient, the model nudges its weights. It pushes the probability of the winner up to 15%, and crushes the probability of the loser down to 5%.

Training vs. Inference

During Training (Learning the Weights):
 We load only two models into the GPU: the active model being updated, and the frozen reference model. We feed them batches of winning and losing responses. **During Inference / Generation (Runtime Chat with ChatGPT/Claude):**
 The reference model is discarded. The newly aligned, single model operates normally, predicting the next word sequentially for the user's prompt.

Real-World NLP Behaviors

- Democratization of Model Alignment:** Because DPO only requires two models in memory instead of four, individual developers can align their own models.
- Format Adherence Enforcement:** In chat systems, DPO effectively suppresses unwanted conversational artifacts (like "Sure! I can help with that" or "I'm sorry, but I can't assist with that request").

7.4 Reasoning RL & Verifiable Rewards (GRPO / DeepSeek-R1 / o1)

The Conceptual Foundation

Both RLHF and DPO rely heavily on human judges to say which answer is better. This works wonderfully for poetry, tone, and summarization, but it's slow and expensive. To fix this, we remove humans entirely. Instead of human taste, we use **Verifiable Rewards**. We ask the model a math question, let it generate a solution, and then we use a separate verifier (or even the model itself in a self-reflection loop) to check if the solution is correct. If correct, we give a high reward; if incorrect, we give a low reward. This turns subjective human preference into objective, verifiable truth.

How GRPO Works in Practice

When a user asks a complex question, we instruct the model to generate a "group" of several different complete attempts. We then run each attempt through a verifier. The verifier outputs a binary result (True/False or 1/0). We calculate the average reward for the group and use that to update the model's weights via a reinforcement learning algorithm like PPO.

xml

Math/Code Q

Sample G Rollouts

o_1: "... ans: 42"

o_2: "... ans: 17"

o_G: "... ans: 42"

Rule-Based Verifiers

r_1 = 1.0 (Correct)

r_2 = 0.0 (Wrong)

...

⊕ MATHEMATICAL WORKING & FORMULATION

First, we need to calculate an "Advantage" score for every single attempt in the group. This tells us not just if an answer was right, but how impressive it is relative to the model's current abilities.

$$A_i = \frac{r_i - \text{mean}(\{r_1 \dots r_G\})}{\text{std}(\{r_1 \dots r_G\}) + \epsilon}$$

In this formula, A_i is the Advantage score for a specific attempt (output i). The variable r_i is the raw verifiable score from our computer checker (usually a 1 for correct, or a 0 for incorrect). From this raw score, we subtract the $\text{mean}(\{r_1 \dots r_G\})$, which is simply the average score of all the attempts in the entire group G . We then divide by $\text{std}(\{r_1 \dots r_G\})$, which stands for standard deviation—a measure of how spread out the scores are. We add a tiny number ϵ (epsilon) to the bottom just to ensure we never mathematically divide by zero. If an attempt is correct while most others in the group failed, its A_i will be a strong positive number. If it failed while others succeeded, its A_i will be negative.

Next, we need a mathematical way to update our model to favor the attempts that had a positive Advantage. We do this by looking at the ratio of how the probabilities have changed.

$$\text{Ratio} = \frac{\pi_\theta(o_i \mid q)}{\pi_{\text{old}}(o_i \mid q)}$$

Here, π_θ represents our new, active model with its shifting dials, generating the specific reasoning output path o_i based on the user's question q . The denominator π_{old} represents the model as it was at the start of the current training step. This ratio simply measures how much our model is amplifying this specific reasoning path. If the ratio is 1.5, we have made this text 50% more likely to occur.

Finally, we combine these pieces into the ultimate GRPO objective function that the computer strives to maximize.

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G} \sum_{i=1}^G [\min(\text{Ratio} \times A_i, \text{clip}(\text{Ratio}, 1 - \epsilon, 1 + \epsilon) A_i) - \beta \mathcal{D}_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})]$$

This formula ties everything together. $\mathcal{L}_{\text{GRPO}}(\theta)$ is the final loss penalty to be minimized. The $\frac{1}{G} \sum_{i=1}^G$ means we are averaging the results across all G attempts in our group.

Inside the brackets, we take our probability Ratio and multiply it by our Advantage score A_i . If the advantage is positive, the model pushes the ratio higher to reap the reward. But notice the $\min$ (minimum) function and the clip function. These act as strict speed limits. If the model tries to increase the ratio too fast (going beyond a safe margin defined by $1 + \epsilon$), the clip function cuts it off, preventing the network from making a drastic, unstable change that could break its understanding of grammar. Finally, just like in RLHF, we subtract our Kullback-Leibler leash penalty ($\beta \mathcal{D}_{\text{KL}}$) tethered to the reference model π_{ref} to ensure the model doesn't devolve into spewing pure gibberish while chasing math rewards.

□ Concrete Numerical Toy Walk-Through

We ask the model to solve an algebra problem. We set our group size $G = 4$. The model tries 4 times.

Attempt 1: Fails (Score = 0)

Attempt 2: Fails (Score = 0)

Attempt 3: Succeeds (Score = 1)

Attempt 4: Fails (Score = 0)

The mean score of the group is 0.25 (since 1 divided by 4 is 0.25).

For the successful Attempt 3, the advantage is positive: $1 - 0.25 = +0.75$.

For the failing Attempts 1, 2, and 4, the advantage is negative: $0 - 0.25 = -0.25$.

The model looks at Attempt 3, identifies the intermediate reasoning steps it took, and nudges its probability dials upward to make those specific logical deductions more likely in the future. It actively learns to suppress the logical mistakes made in the other three attempts.

⚙ Training vs. Inference

• During Training (Learning the Weights):

The model is asked a complex question and forced to rapidly sample dozens of full, step-by-step reasoning outputs. The deterministic code-checkers evaluate the final answers. The model calculates the group average, computes the advantage scores, and backpropagates the gradient to reinforce the logical paths that survived the automated checkers.

• During Inference / Generation (Runtime Chat with ChatGPT/Claude):

When a user asks a question, models trained with GRPO (like OpenAI's o1 or DeepSeek-R1) do something completely unique: they generate thousands of hidden "thinking tokens" before answering. The model writes out its inner monologue, testing ideas, backtracking, and verifying its own logic autonomously, mimicking the exact step-by-step reasoning pathways reinforced during training, before finally outputting the crisp final answer to the user.

🌐 REAL-WORLD & NLP BEHAVIORS

1. The "Aha Moment" (Autonomous Self-Correction): During the pure reinforcement learning training of models like DeepSeek-R1-Zero, researchers observed something astounding: models spontaneously started generating phrases like *"Wait, let me double check my previous equation..."* and correcting their own errors mid-generation, despite never being trained on human reasoning templates! The RL pressure organically taught them that self-doubt leads to higher verified scores.

2. Competitive Programming & Math Olympiads: Verifiable RL allows models to jump from roughly 20% accuracy to over 90% accuracy on fiercely competitive coding challenges (like Codeforces) and high-school math olympiads (AIME). They achieve this purely by learning how to mentally self-verify and debug their own code in their hidden reasoning streams before emitting the final answer.